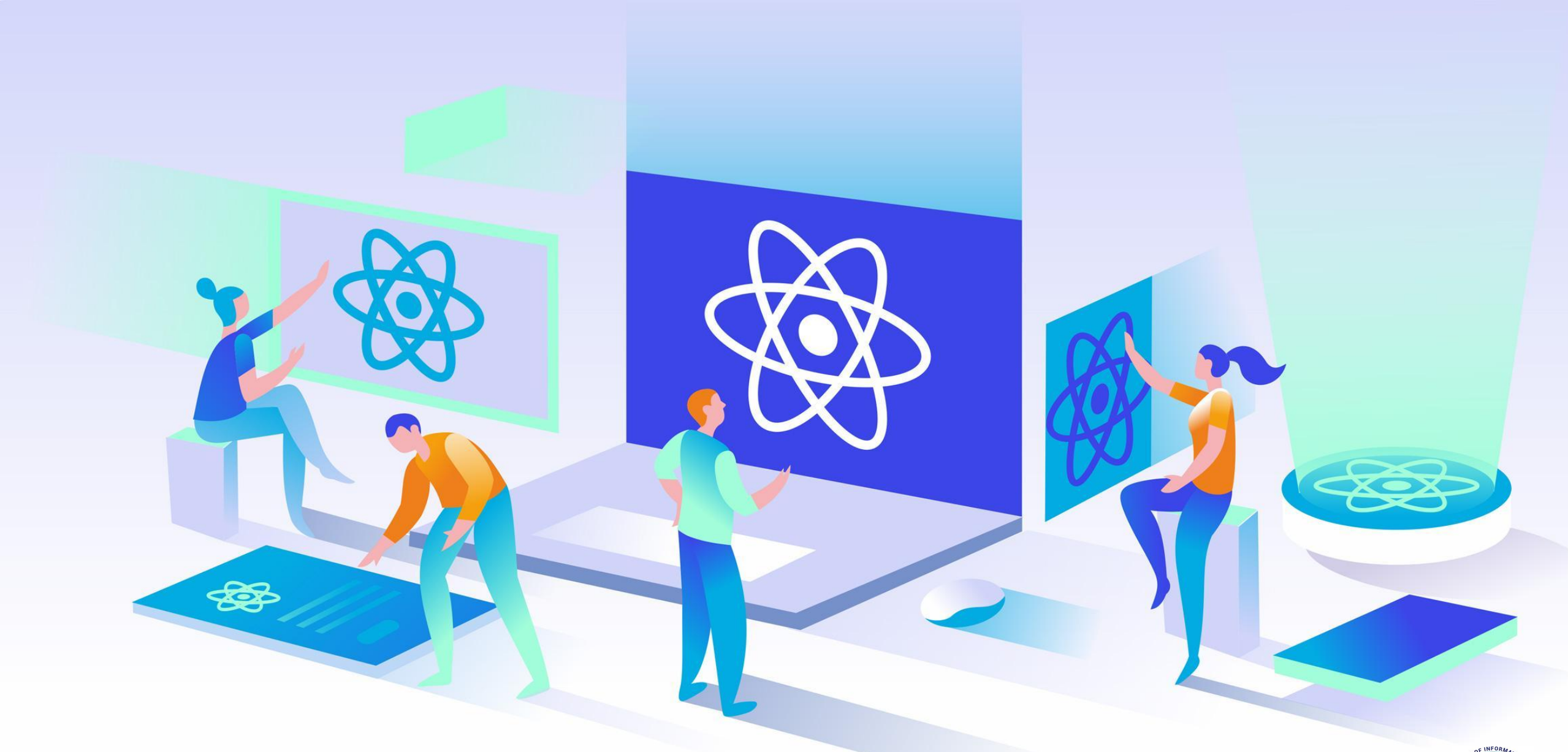


Programming in React JS

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY
jiit
विद्या तत्व ज्योतिसमः

React

- React is a JavaScript library used for building user interfaces
- It is developed and maintained by Facebook
- It is not a framework like (Angular, Vue)
- Why React is in demand?
  - One can use external libraries
- Advantages of external : users have option to customize or download libraries according to application or project
- Angular and Vue are bundled and have many things which the user's may or may not want to use
- React is light weight

Single Page Application (SPA)

- A Single Page Application (SPA) is a web application that interacts with the user by dynamically rewriting the current web page with new data from the web server, instead of the default method of the browser loading entire new pages from the server.[1]

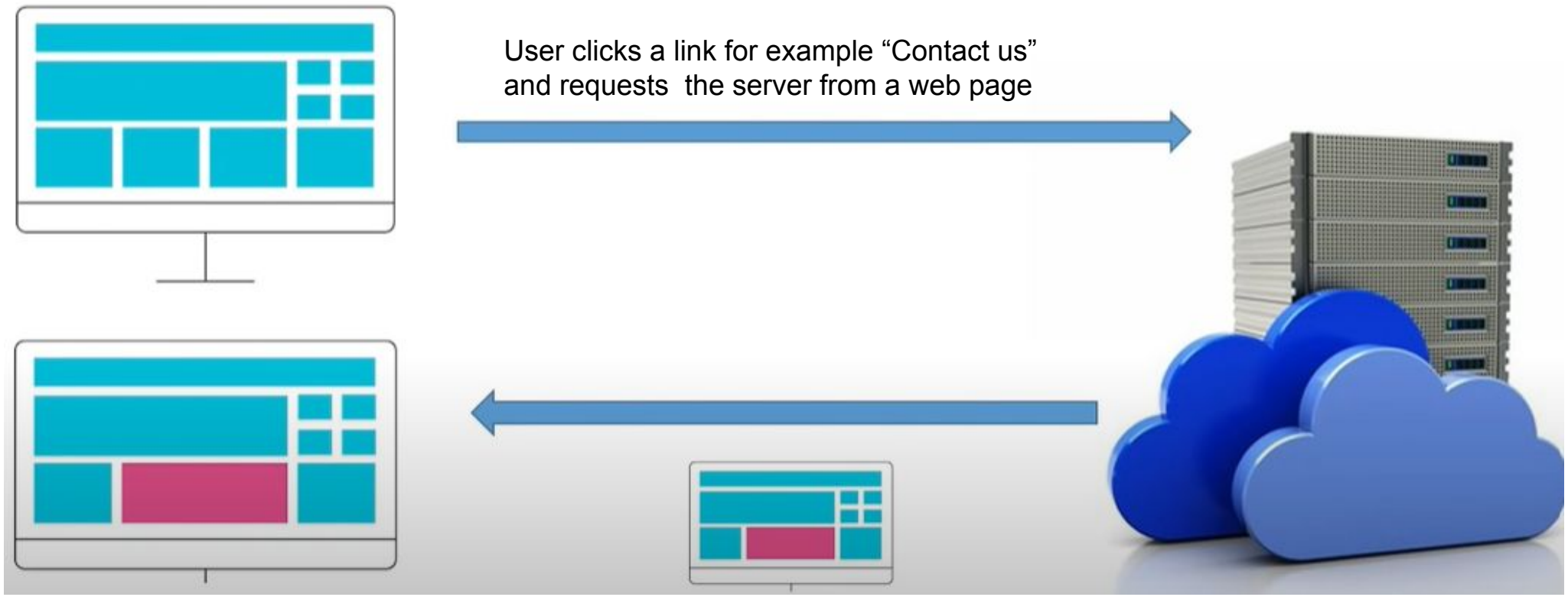
- All required HTML, JS, and CSS code is retrieved by the browser with a single page load. Thereafter, based on user request additional resources are dynamically loaded and added to the page.

- Markup and data are requested independently

- Aims at providing seamless UX - no page reloads, no extra wait time, uninterrupted UX between successive pages, making the application behave more like a native application

Text Courtesy: [1] https://en.wikipedia.org/wiki/Single-page_application

Multi page app



User clicks a link for example “Contact us” and requests the server from a web page

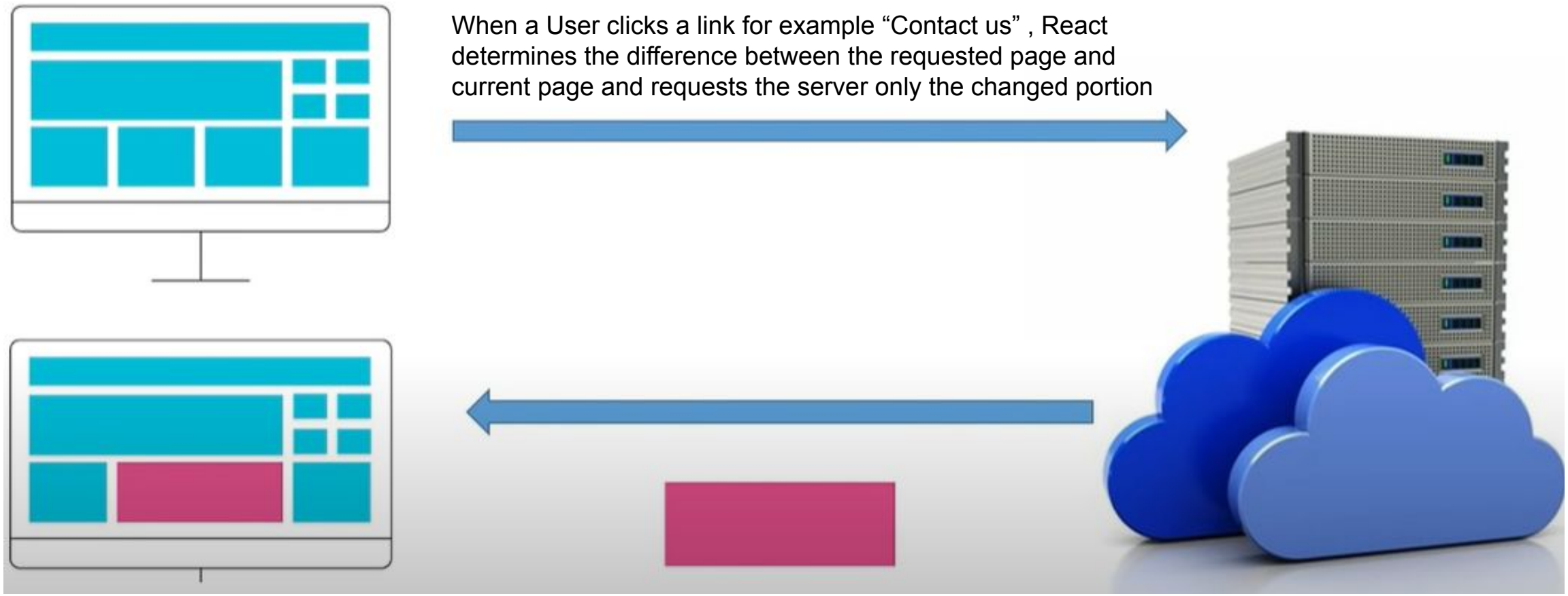
Server creates and sends the new page “Contact Us”

Entire page is created

Browser’s engine renders the page (recreated)

How SPA works

When a User clicks a link for example “Contact us” , React determines the difference between the requested page and current page and requests the server only the changed portion



Browser’s engine does not renders the page, React only changes the required component

Server only sends the updated part instead of completed page

Pros & Cons of SPA

Pros

- Fast, as most resources (HTML, CSS, Scripts) are only loaded once and only data is transmitted back and forth
- Enhanced User Experience
- Highly Cached Application
- Decoupled Front and Back-end (no shared resources)
- Less load on server
- Debugging becomes simpler, especially within the Chrome browser's inspection function

Cons

- Slow speed of initial load
- JavaScript dependency in browser
- Less secure due to Cross-Site Scripting (XSS)
- Memory leak in JS can cause system slow down
- SEO is tricky
- Difficult to analyze user behavior
- No browser history

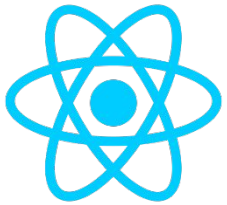
When to use SPA and when not

	Use SPA	Don’t use SPA
Want a rich interaction between the user and your application	✓	✗
Want to provide real-time updates on the page	✓	✗
Small / Mid-size applications involving small data volumes	✓	✗
Application content is purely static (like a static website)	✗	✓
SaaS platforms, social networks, and closed communities where SEO doesn’t matter	✓	✗
Enterprise-based application involving complex requirements	✗	✓

SPA libraries/ Frameworks

Some frameworks/ libraries that can be used to create single-page applications

ANGULAR



REACT

VUE

EMBER

POLYMER

BACKBONE

AURELIA

KNOCKOUT

METEOR

MVC Design Pattern

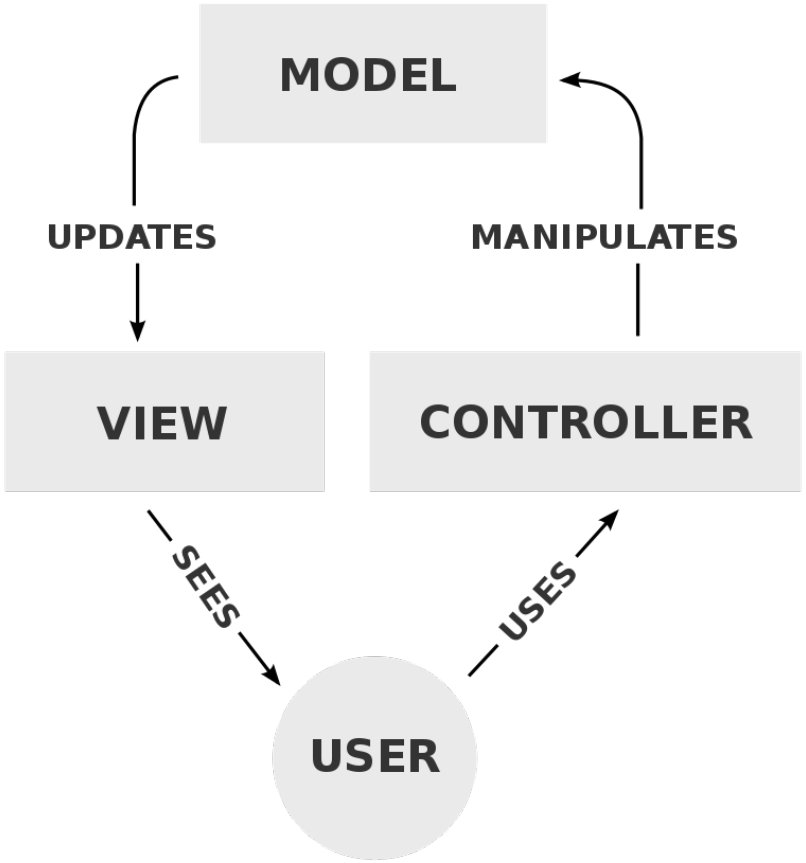


Image Courtesy:
https://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93controller

Model-View-Controller (MVC) design pattern is used for developing applications in a manner that divides the related program logic into three interconnected elements - a data model, presentation information, and control information.

Model – Responsible for managing the data of the application.

It receives user input from the controller.

View – Responsible for all the UI logic of the application required for representation of information.

Multiple views of the same information are possible.

Controller - Responsible for controlling the application logic

It acts as the coordinator between the View and Model

It accepts user input and converts it to commands for the model or view.

How MVC works

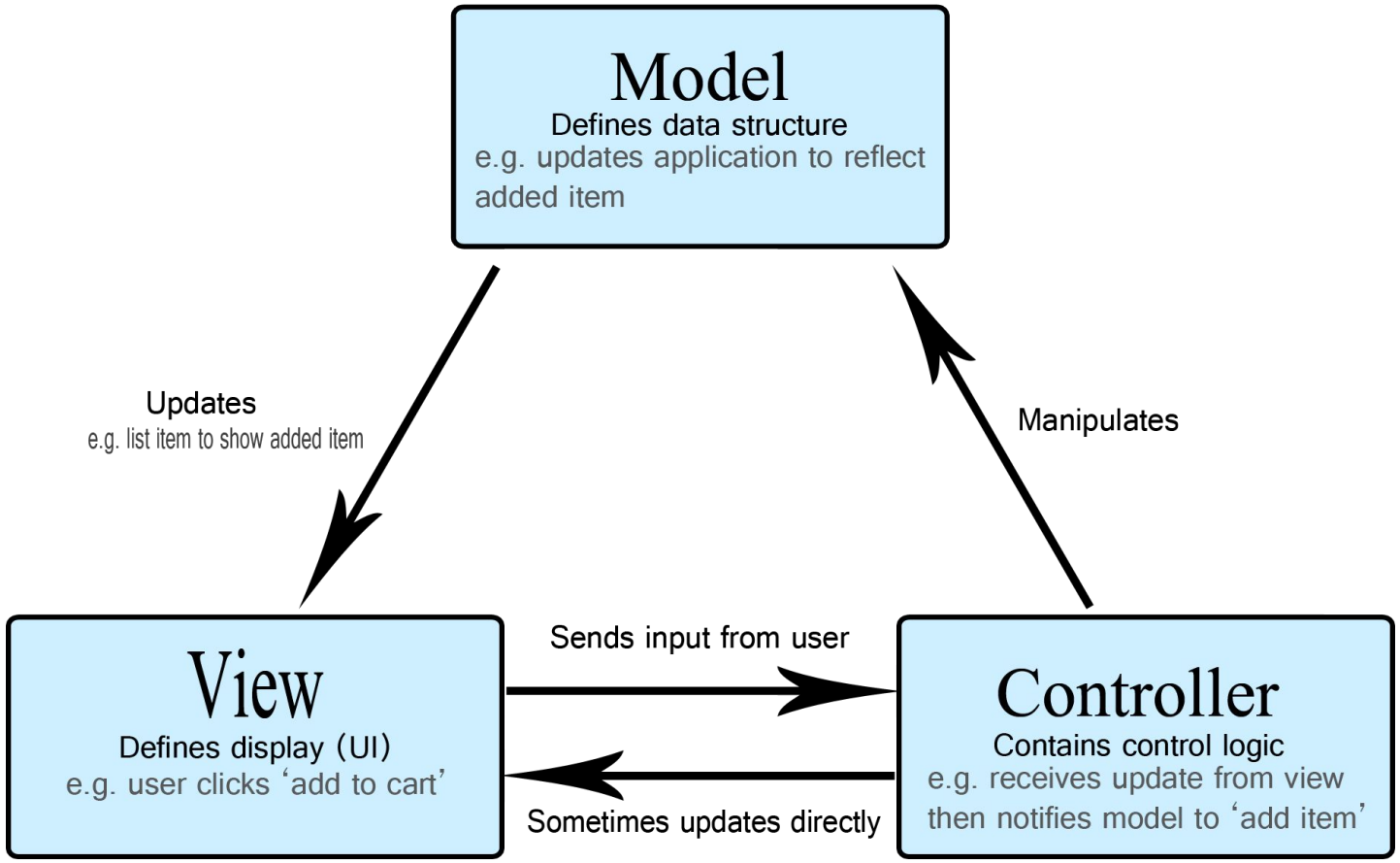


Image Courtesy: https://blog.cloudboost.io/what-is-model-view-controller-124a9942246

Ex. – Shopping List App

Model – specifies what data the list items should contain like item name, price, etc.

View – defines how the list is presented to the user, and receives data from the model to display.

Controller –

Add/ delete item - Requires the model to be updated, so the input is sent to the controller, which manipulates the model and the model then sends updated data to the view.

Sort item on name/ price – controller directly updates the view without updating the model

Full Stack Development

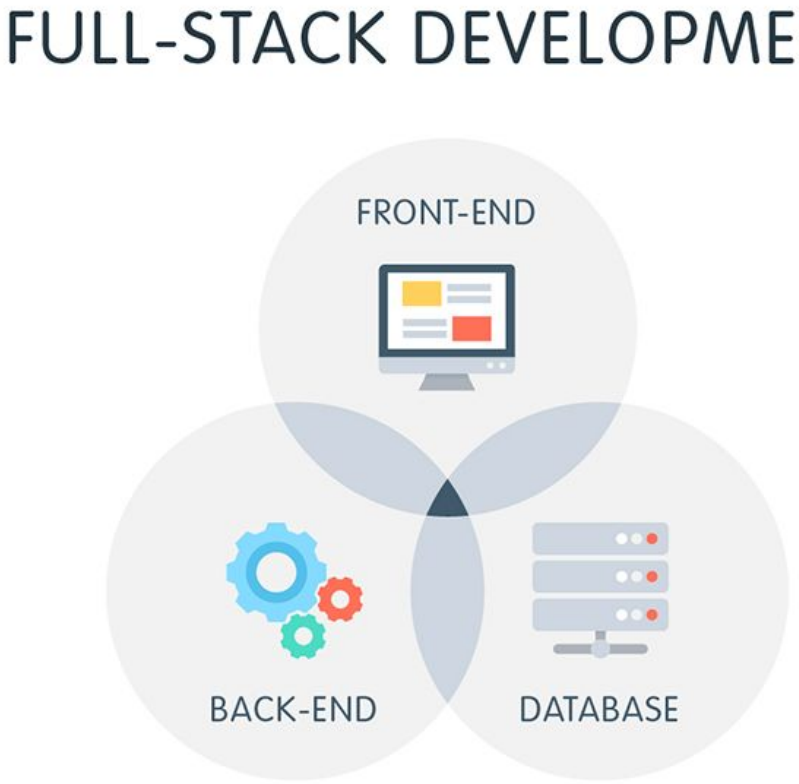
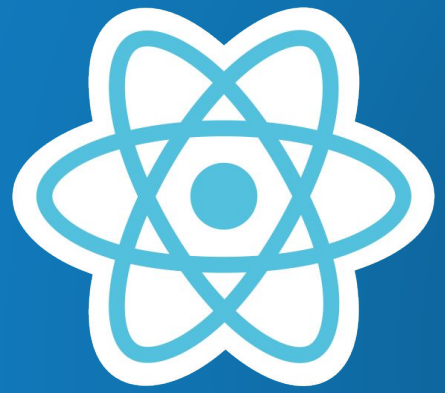


Image Courtesy:
https://medium.com/@sepandassadi/become-a-full-stack-web-developer-free-resources-8a1c2c0ebd41

The combination of multiple technologies used to develop any web application is called a **STACK**

- **LAMP** – Linux, Apache, MySQL, PHP
- **MEAN** – MongoDB, Express, AngularJS, Node.js
- **MERN** – MongoDB, Express, React, Node.js



React

- React was created by Jordan Walke, a software engineer at Facebook.
- It is a JavaScript library that aims to simplify development of visual interfaces.
- It is used for building fast and interactive user interfaces for web and mobile applications.
- It is an open-source, component-based, front-end library responsible only for the application's view layer.

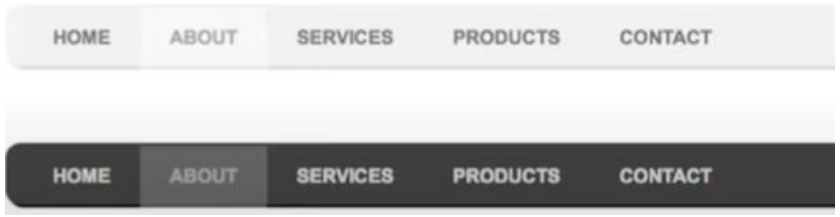
Why React

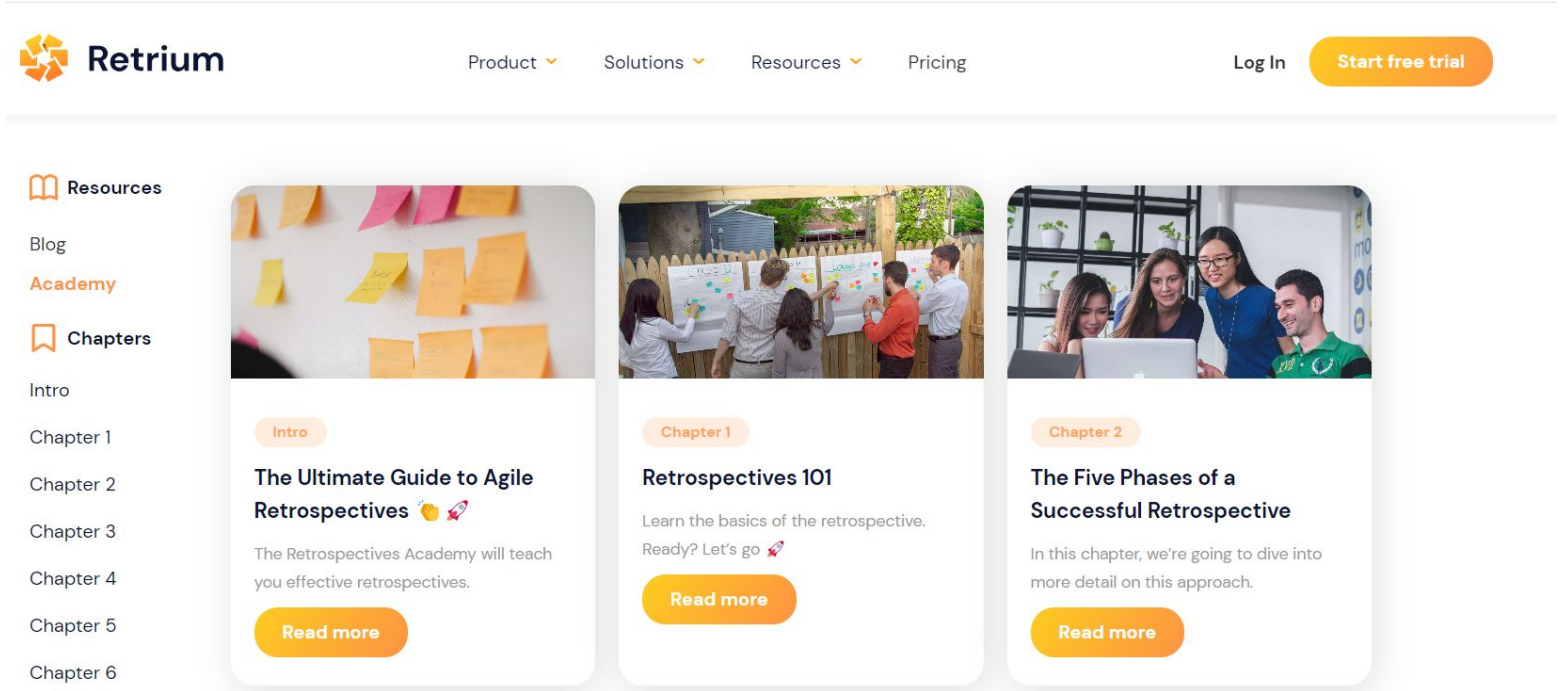
- **Easy creation of dynamic applications** - A component-based approach, well-defined lifecycle, and use of just plain JavaScript make React very simple to learn, and build dynamic web and mobile applications.
- **Fast render with Virtual DOM** - Virtual DOM updates only the items in the Real DOM that were changed, instead of updating all of the components again, as conventional web applications do.
- **Reusable components** - Components are the building blocks of any React application. A single app usually consists of multiple components that can be reused throughout the application, which in turn reduces the application's development time.
- **Unidirectional data flow** - React follows a unidirectional data flow, wherein data always flows from parent to child component. This unidirectional data flow assists in debugging errors and identification of problematic points in an application.
- **Small learning curve** - React is easy to learn, as it mostly combines basic HTML and JavaScript concepts with some beneficial additions.
- **Dedicated tools for easy debugging** - Facebook has released a Chrome extension that can be used to debug React applications. This makes the process of debugging React web applications faster and easier.

Text Courtesy: https://www.simplilearn.com/tutorials/reactjs-tutorial/what-is-reactjs

React Components

- Components are reusable building block in user interface (html and CSS with JS make a component)
- Benefits: Reusability and helps in maintain code
- In react we build components with html and CSS with JS and then by combining them we built a webpage





Installation Steps

- Website: https://reactjs.org/docs/getting-started.html
- Click on Create React App (details the prerequisite)

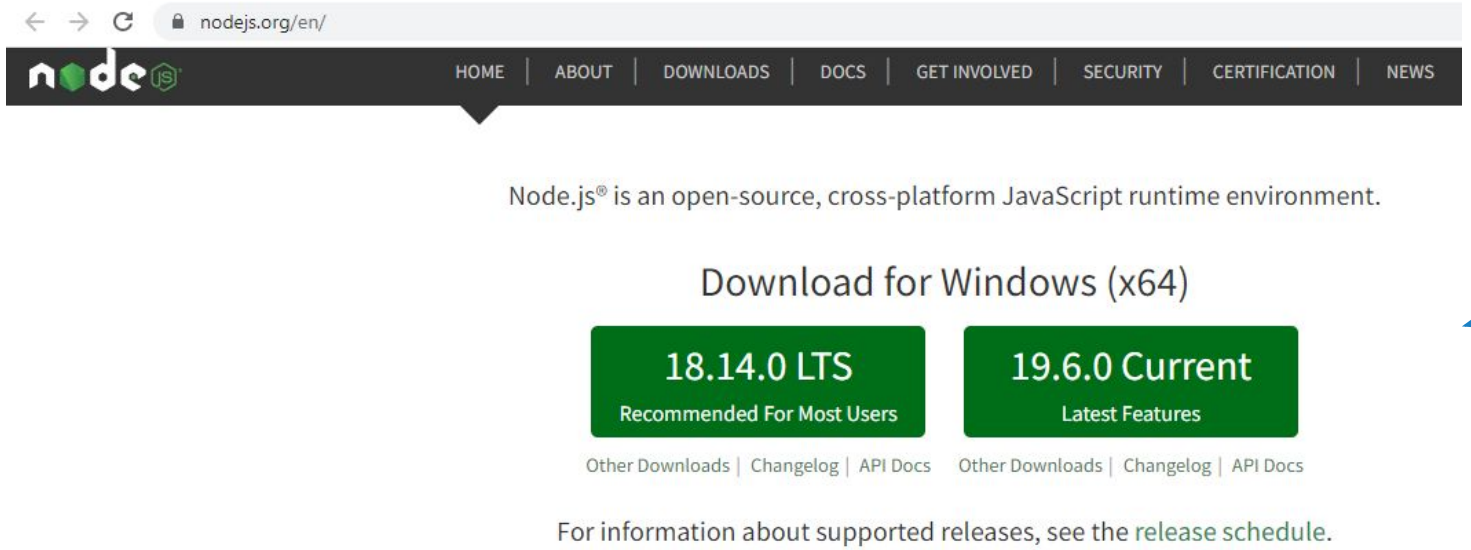
It sets up your development environment so that you can use the latest JavaScript features, provides a nice developer experience, and optimizes your app for production. You'll need to have Node >= 14.0.0 and npm >= 5.6 on your machine. To create a project, run:

```
npx create-react-app my-app
cd my-app
npm start
```

INSTALLATION

Getting Started
Add React to a Website
Create a New React App
CDN Links

- Download Node JS: https://nodejs.org/en/



Node JS

- Node.js is a cross-platform, open-source server environment that can run on Windows, Linux, Unix, macOS, and more.
- Node.js is a back-end JavaScript runtime environment and library, runs on the V8 JavaScript Engine, and executes JavaScript code outside a web browser.
- JS was developed to run on browsers (Google chrome, Firefox)
- Chrome has JS engine (V8), this engine runs JS on chrome
- V8 was updated to run on Computer like C, C++
- Node. js can be used on the frontend as well as the backend.
- Node .js is used to create react apps

To check for successful installation of Node js

- Open command prompt and type node

```
C:\>node
Welcome to Node.js v19.6.0.
Type ".help" for more information.
```

- Check whether JavaScript can be executed

```
> 2+2
4
> 4+6
10
>
(To exit, press Ctrl+C again or Ctrl+D or type .exit)
>
```

Check for necessary tool

- Type npm

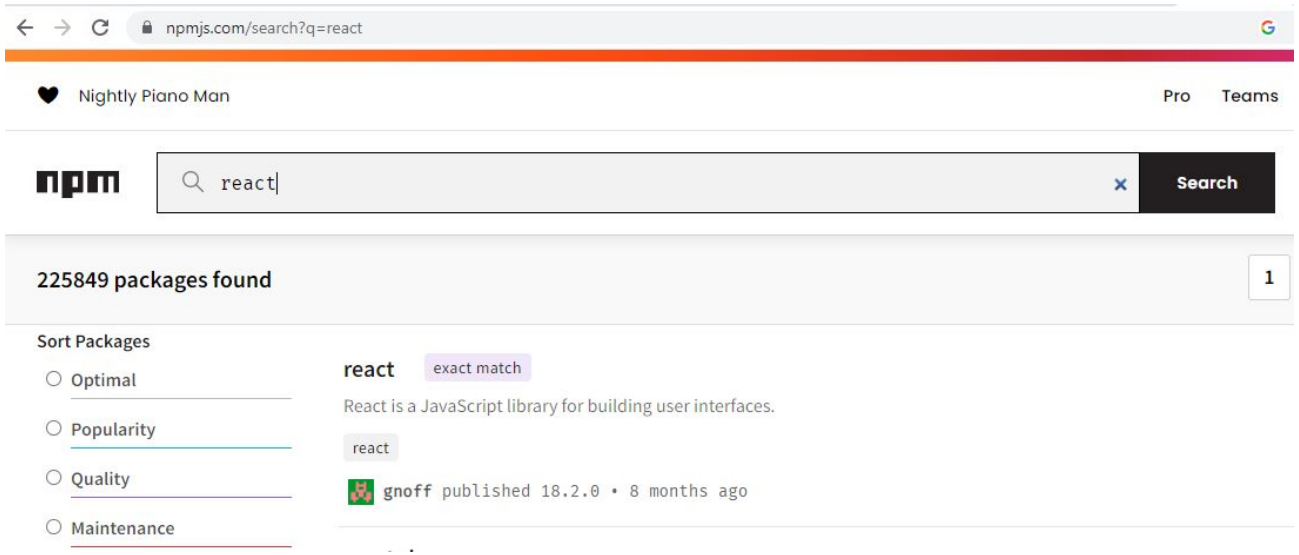
```
C:\>npm
npm <command>

Usage:

npm install        install all the dependencies in your project
npm install <foo>  add the <foo> dependency to your project
npm test           run this project's tests
npm run <foo>      run the script named <foo>
npm <command> -h   quick help on <command>
npm -l             display usage info for all commands
npm help <term>    search for help on <term> (in a browser)
npm help npm       more involved overview (in a browser)

All commands:
```

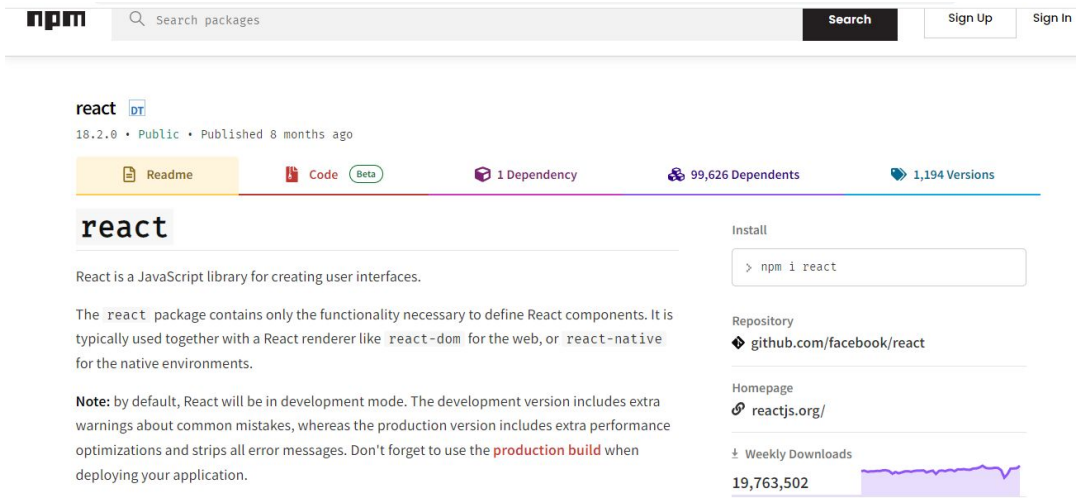
- Type npx

```
C:\>npx

Entering npm script environment at location:
C:\
Type 'exit' or ^D when finished

Microsoft Windows [Version 10.0.19043.1766]
(c) Microsoft Corporation. All rights reserved.

C:\>cd Desktop
The system cannot find the path specified.

C:\>dir
 Volume in drive C is OS
 Volume Serial Number is 0A0E-92FC
```

NPM(**Node Package Manager**) and NPX (Node Package Execute)

- npm is the world's largest software registry. Open source developers from every continent use npm to share and borrow packages.
- It is a package manager
- Example: Facebook created React package manager so that the users can develop websites. Instead of giving download link to users, Facebook uploaded all files which were necessary to develop a react app on npm server in form of package (folder).
- NPM gives us commands so that we can download files from npm server on our systems
- NPX is npm package runner that can execute any package that you want from the npm registry

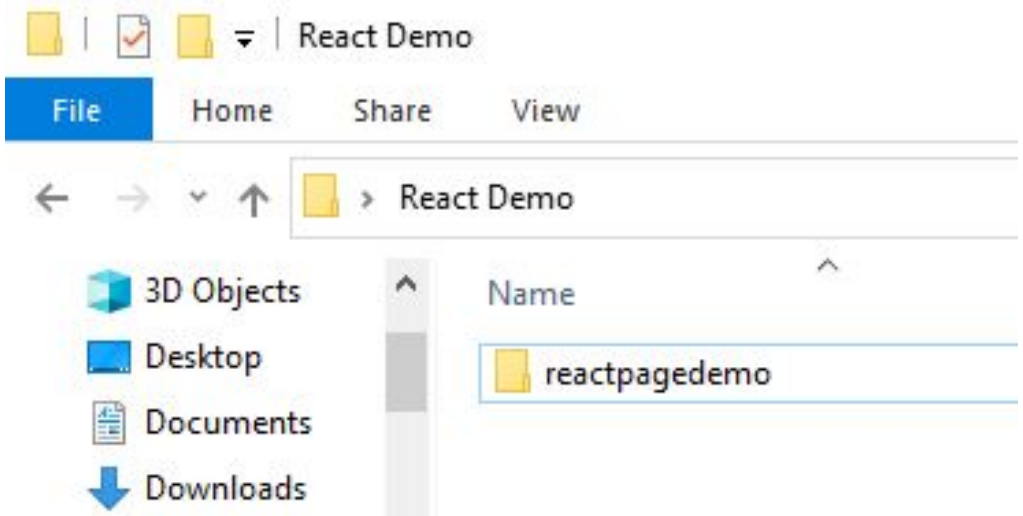


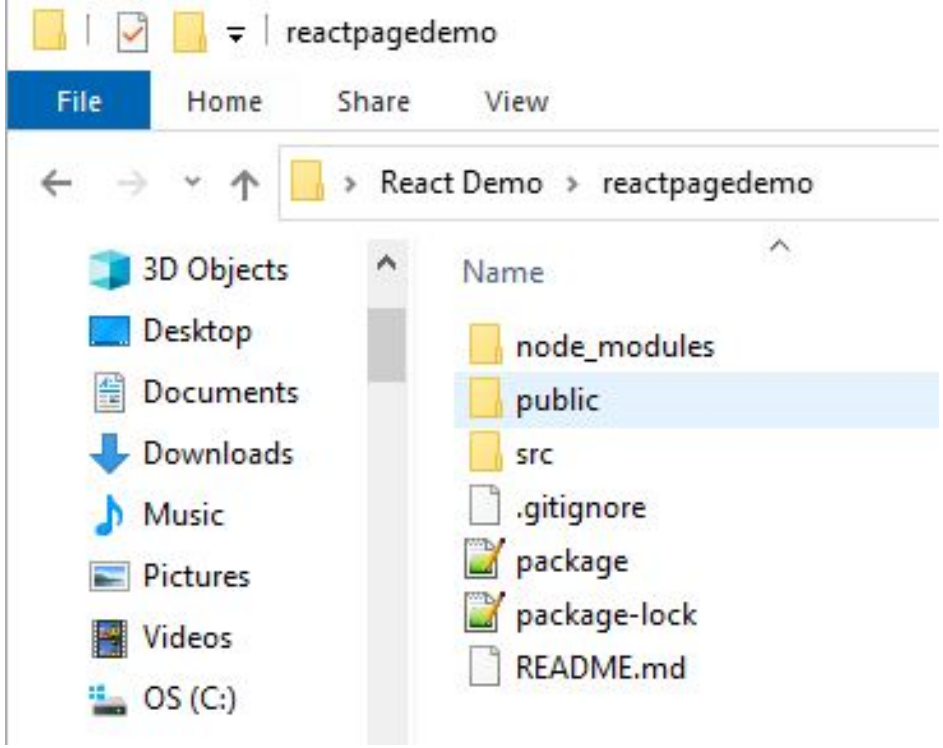


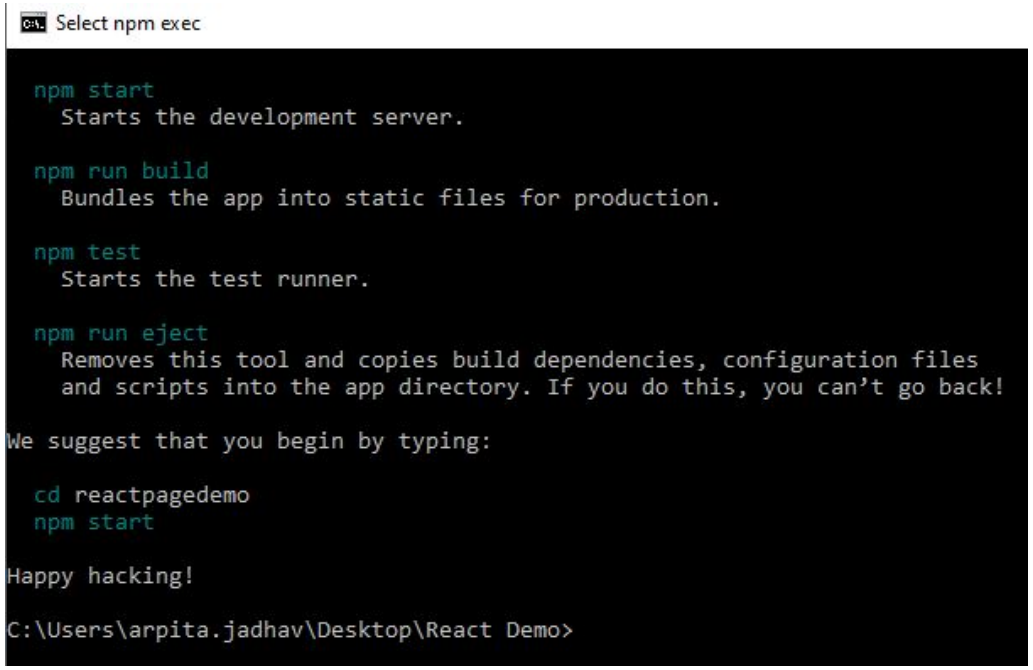
After installing node

- Create a folder
- Open command prompt and navigate to the folder
- Execute the command :
- **npx create-react-app reactpagedemo**
- This command installs the packages to create app from npm server and saves in the folder created above

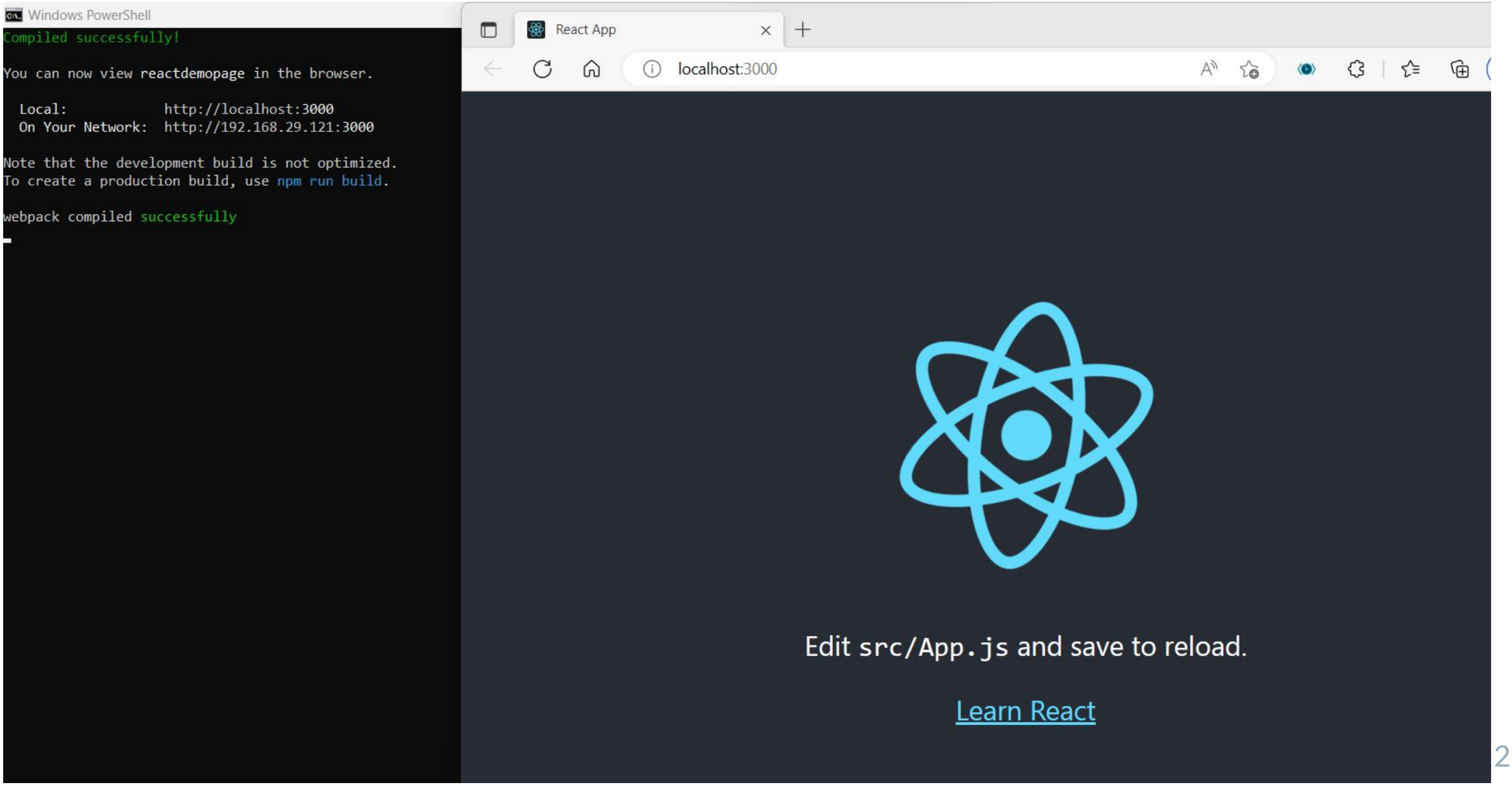
```
C:\Users\arpita.jadhav\Desktop\React Demo>npx create-react-app reactpagedemo

Creating a new React app in C:\Users\arpita.jadhav\Desktop\React Demo\reactpagedemo.

Installing packages. This might take a couple of minutes.
Installing react, react-dom, and react-scripts with cra-template...


added 1416 packages in 6m

231 packages are looking for funding
  run `npm fund` for details
Git repo not initialized Error: Command failed: git --version
    at checkExecSyncError (node:child_process:885:11)
    at execSync (node:child_process:957:15)
    at tryGitInit (C:\Users\arpita.jadhav\Desktop\React Demo\reactpagedemo\node_modules\react-s
:5)
    at module.exports (C:\Users\arpita.jadhav\Desktop\React Demo\reactpagedemo\node_modules\rea
s:276:7)
```


React Demo
File
Home
Share
View
React Demo
3D Objects
Desktop
Documents
Downloads
Name
reactpagedemo

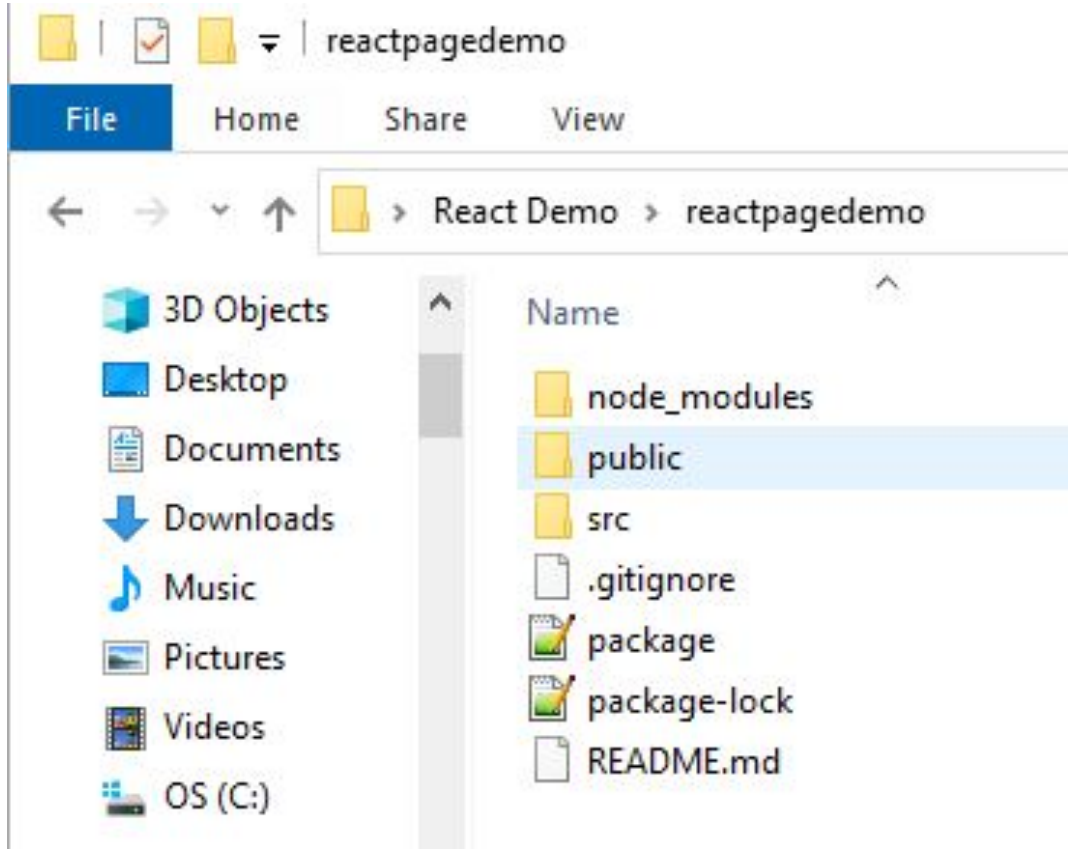

reactpagedemo
File
Home
Share
View
React Demo
reactpagedemo
3D Objects
Desktop
Documents
Downloads
Music
Pictures
Videos
OS (C:)
Name
node_modules
public
src
.gitignore
package
package-lock
README.md


Select npm exec
npm start
Starts the development server.
npm run build
Bundles the app into static files for production.
npm test
Starts the test runner.
npm run eject
Removes this tool and copies build dependencies, configuration files
and scripts into the app directory. If you do this, you can't go back!
We suggest that you begin by typing:
cd reactpagedemo
npm start
Happy hacking!
C:\Users\arpita.jadhav\Desktop\React Demo>

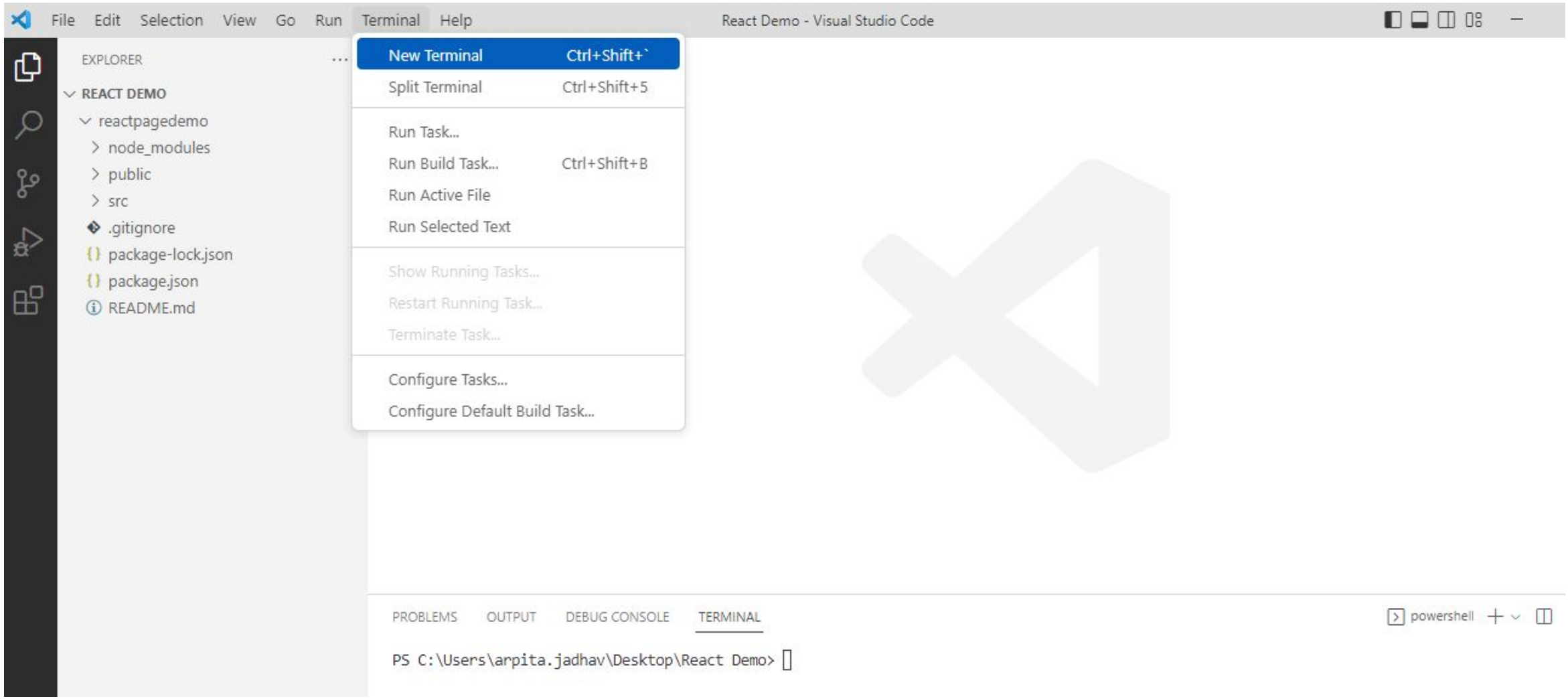
Navigate to the app
Type npm start, it will open the browser



Open the folder in VS code



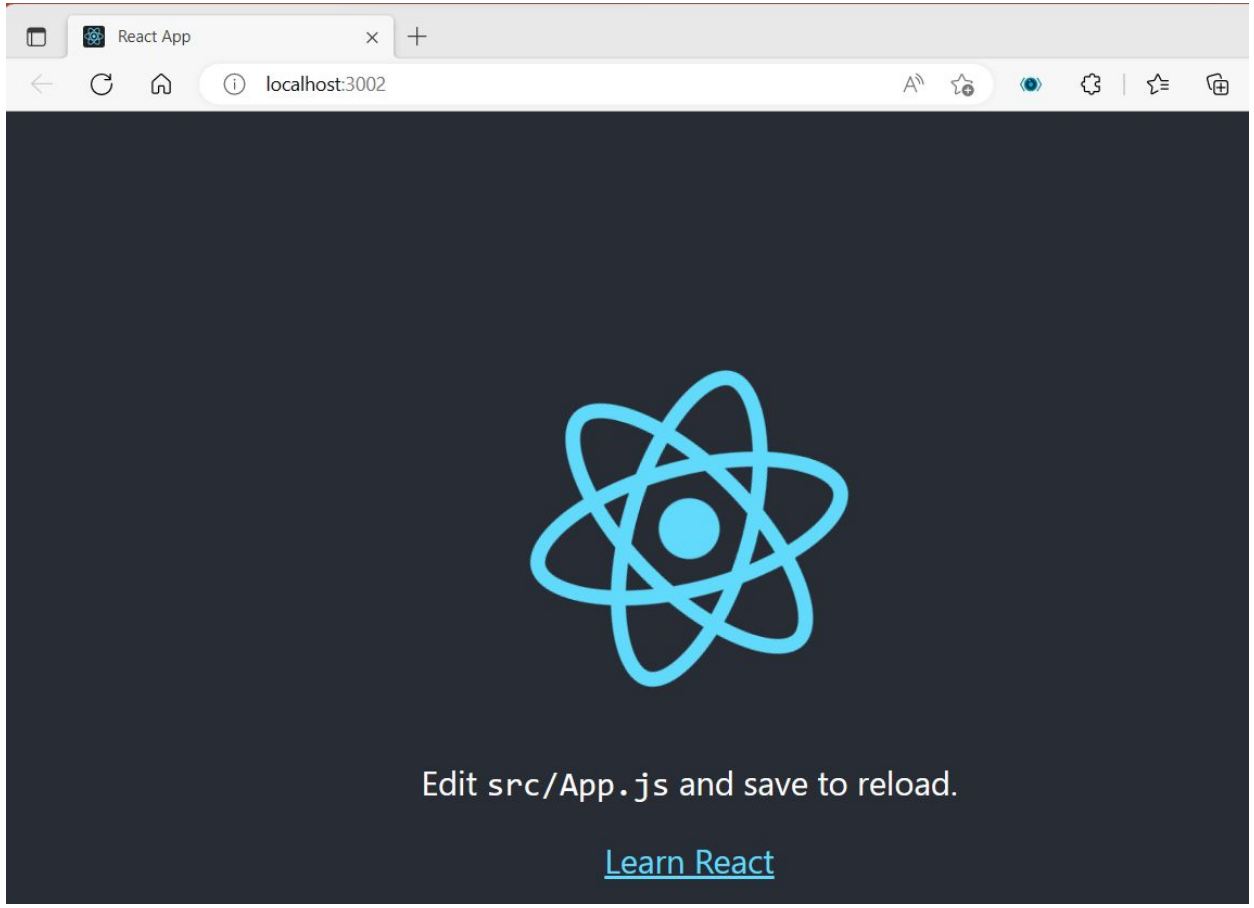
Open Terminal in VS code



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\Users\abhin\Desktop\React Demo> cd .\reactdemopage\
PS C:\Users\abhin\Desktop\React Demo\reactdemopage> npm start
```

```
Compiled successfully!

You can now view reactdemopage in the browser.

  Local:            http://localhost:3002
  On Your Network:  http://192.168.29.121:3002

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```



Basic react app/website/sample project will run, a server will be created .

Note : server should remain on, so that the changes are displayed successfully

Getting Started

Prerequisites

- Familiarity with HTML & CSS.
- Basic knowledge of JS and programming.
- Basic understanding of the DOM.
- Familiarity with ES6 syntax and features.
- Node.js and npm installed globally

Write React code directly in HTML

- Include three CDN's in your HTML file:

  React - the React top level API

  React-DOM - adds DOM-specific methods

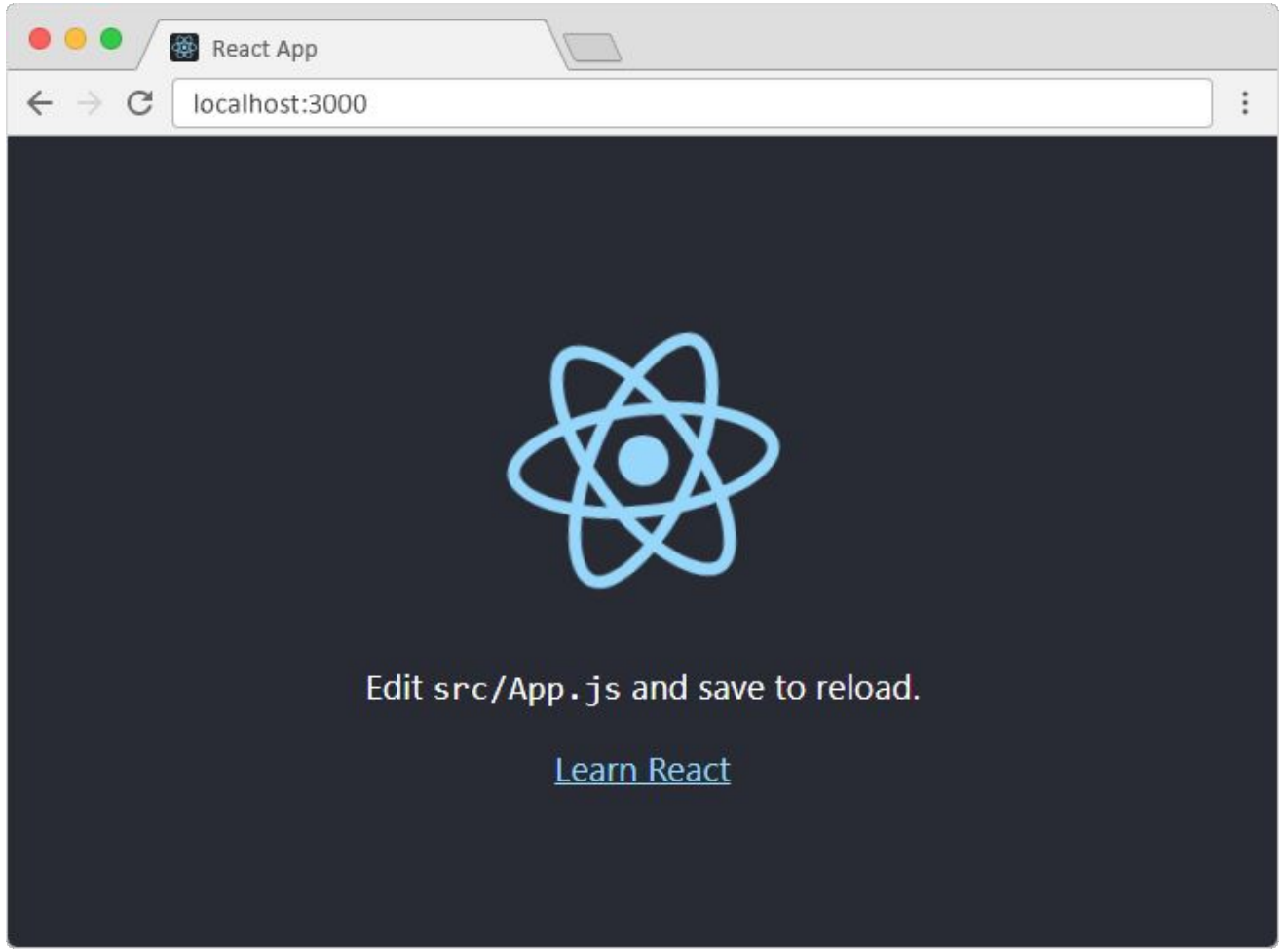
  Babel - a JavaScript compiler

```
<!DOCTYPE html>
<html>
  <head>
    <script
src="https://unpkg.com/react@17/umd/react.development.js"></script>
    <script
src="https://unpkg.com/react-dom@17/umd/react-dom.development.js"></
script>
    <script
src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  </head>
  <body>

    <div id="root"></div>

    <script type="text/babel">
      class App extends React.Component {
        render() {
          return <h1>Hello world!</h1>
        }
      }

      ReactDOM.render(<App />, document.getElementById('root'))
    </script>
  </body>
</html>
```

Getting Started

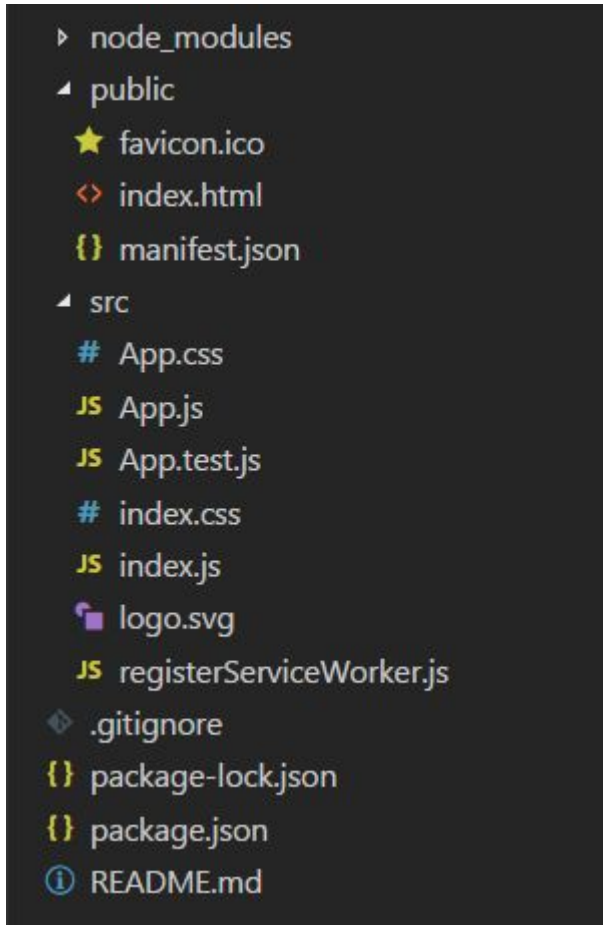
Setting up a React environment – using Create React App

- *Install Node.js*
- *Set up create-react-app* → run command npm install -g create-react-app in your terminal
- *Create a react app* → run command npx create-react-app your-app-name in your terminal
- *Move to the newly created directory*
- *Start the project* → run command npm start in your terminal
- *A new window will open in the web browser at localhost:3000 with your new React app*



Getting Started

When we create a React project, our application's structure looks like this



node_modules - holds all the dependencies and sub-dependencies of our project

Public folder - this folder contains 3 files

- favicon.ico - contains an icon which displays on the browser.
- index.html - due to this file, the public folder, known as “root folder”, gets served by the web server.
- manifest.json – contains metadata about our application.

src folder - this folder contains actual source code for developers. Pre-existing files are:

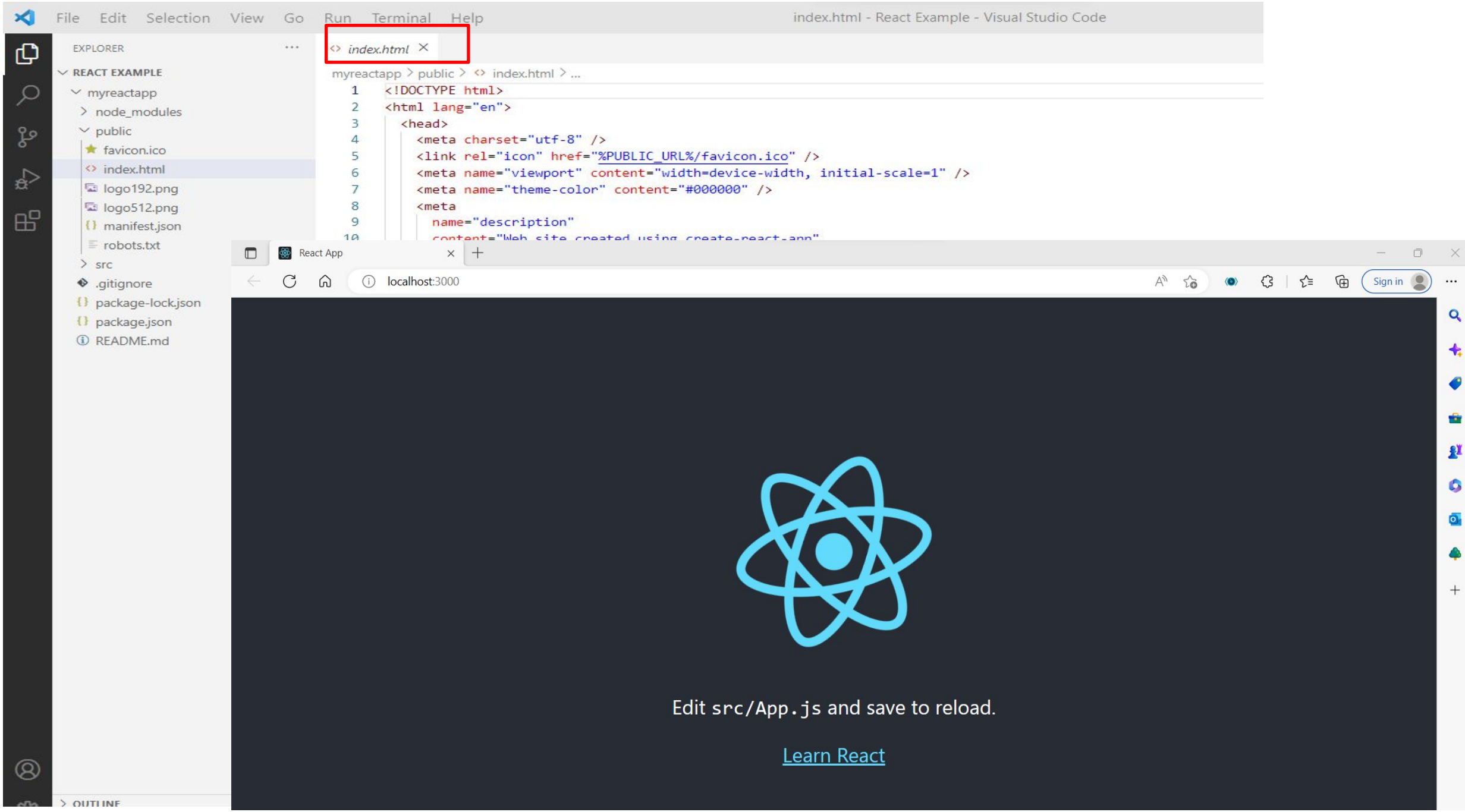
- App.css - gives some CSS classes/ styling which is used by App.js file.
- App.js - a sample React component called “App” which we get for free when creating a new app.
- App.test.js - a test file
- index.css - stores the base styling for the application.
- index.js - stores the main Render call from ReactDOM. It imports App.js component and tells React where to render it.
- logo.svg - contains the logo of Reactjs, visible on the browser.
- registerServiceWorker.js - used for registering a service

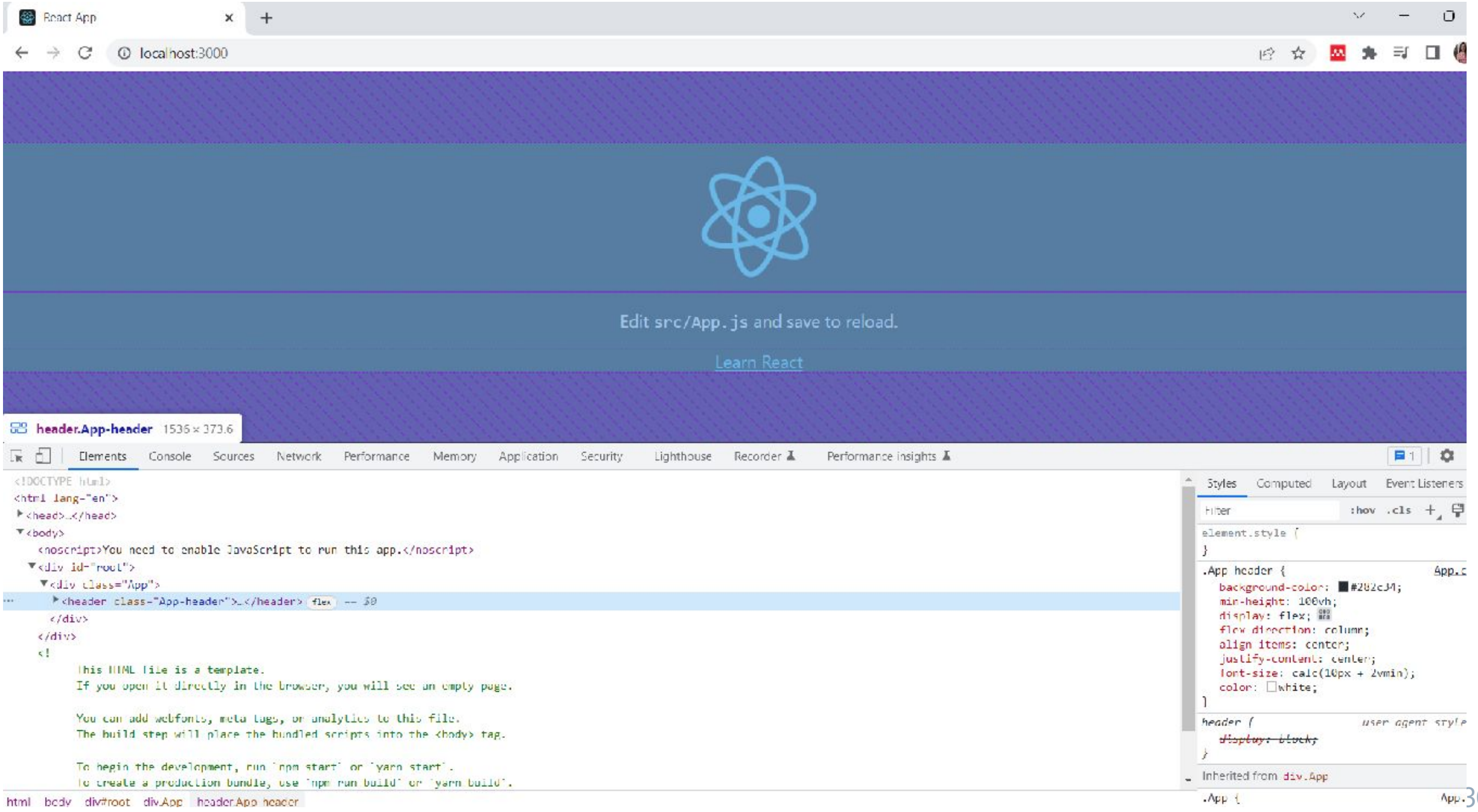
Package.json - contains the information like the name of project, versions, dependencies etc. It stores meta data, all important data about project/app.

Package.lock.json: will install the exact latest version of that package in your application and save it in package.json.

Whenever we install a third party library, it automatically gets registered into this file

Image & Text Courtesy: https://www.c-sharpcorner.com/article/folder-structure-of-react-application/

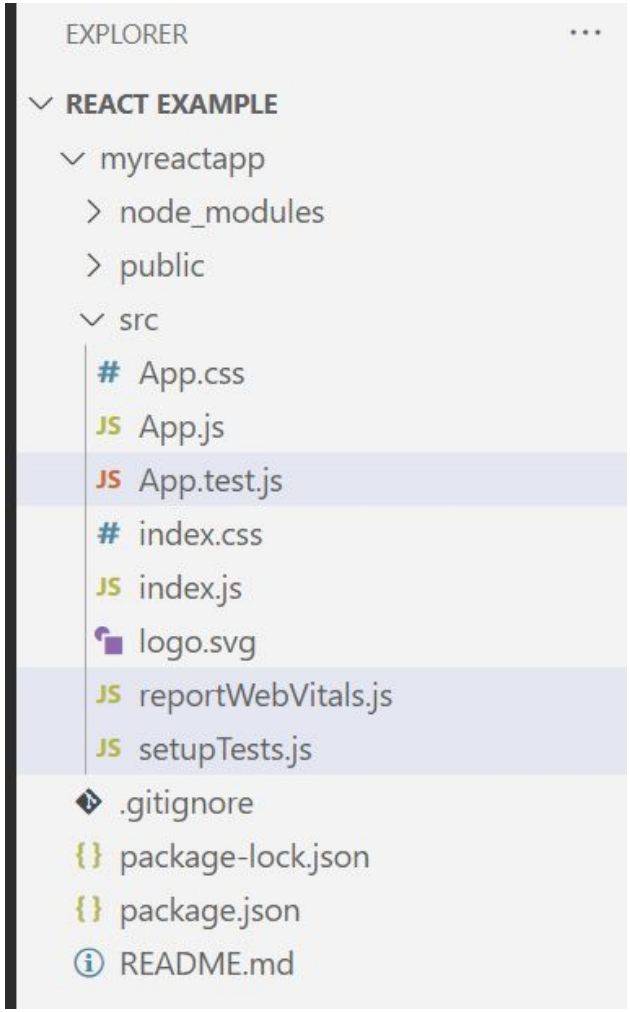

File Edit Selection View Go Run Terminal Help
index.html - React Example - Visual Studio Code
EXPLORER
REACT EXAMPLE
myreactapp
node_modules
public
favicon.ico
index.html
logo192.png
logo512.png
manifest.json
robots.txt
src
.gitignore
package-lock.json
package.json
README.md
index.html
myreactapp > public > index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8" />
5 <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
6 <meta name="viewport" content="width=device-width, initial-scale=1" />
7 <meta name="theme-color" content="#000000" />
8 <meta
9 name="description"
React App
localhost:3000
Sign in
Edit src/App.js and save to reload.
Learn React
OUTLINE


React App
localhost:3000
Edit src/App.js and save to reload.
Learn React
header.App-header 1536 × 373.6
Elements
Console
Sources
Network
Performance
Memory
Application
Security
Lighthouse
Recorder
Performance insights
<!DOCTYPE html>
<html lang="en">
<head>…</head>
<body>
<noscript>You need to enable JavaScript to run this app.</noscript>

<header class="App-header">…</header> flex == $0

<!
This HTML file is a template.
If you open it directly in the browser, you will see an empty page.
You can add webfonts, meta tags, or analytics to this file.
The build step will place the bundled scripts into the <body> tag.
To begin the development, run `npm start` or `yarn start`.
To create a production bundle, use `npm run build` or `yarn build`.
html body div#root div.App header.App-header
Styles
Computed
Layout
Event Listeners
Filter
:hov .cls
element.style {
}
.App header {
background-color: #282c34;
min-height: 100vh;
display: flex;
flex-direction: column;
align-items: center;
justify-content: center;
font-size: calc(10px + 2vmin);
color: white;
}
header {
user agent style
display: block;
}
Inherited from div.App
.App {

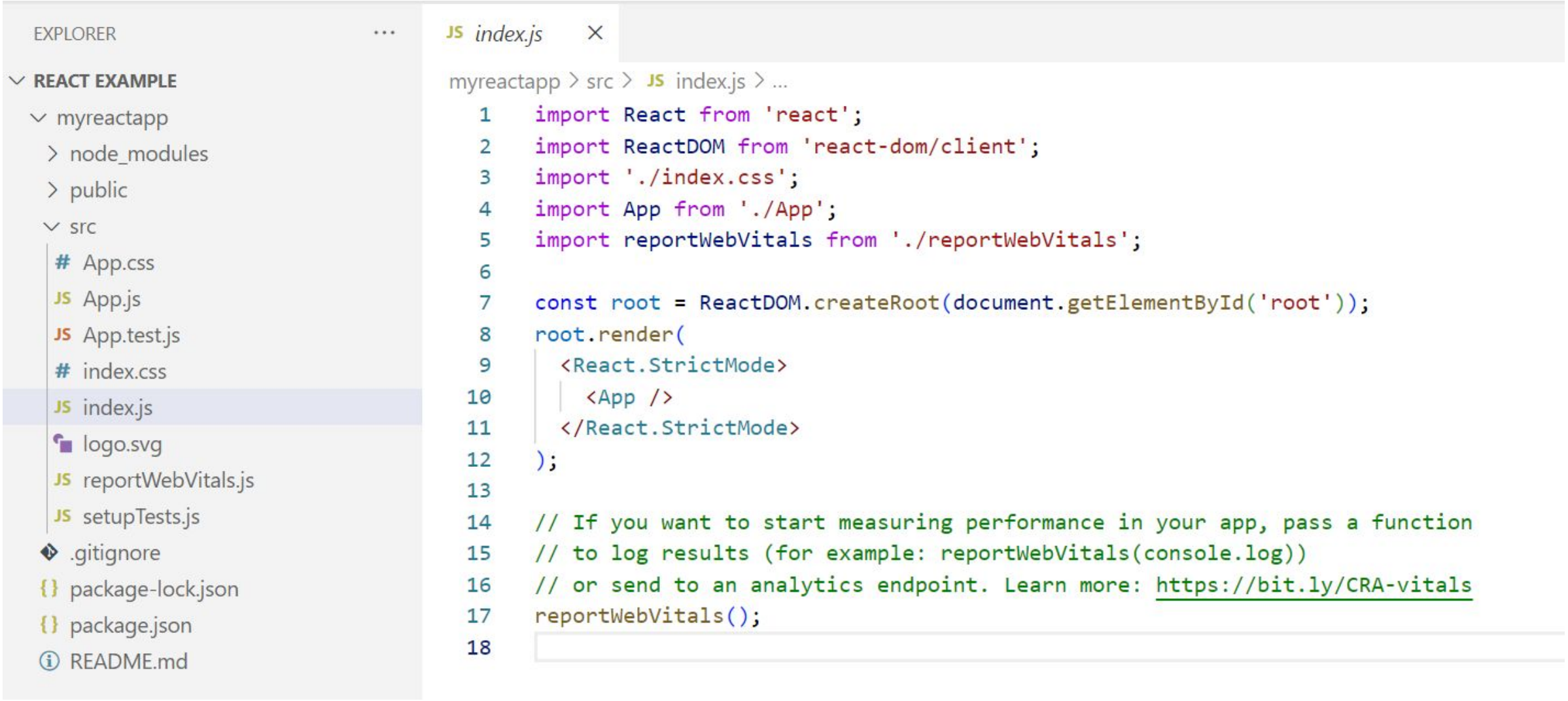
Inside Source Folder



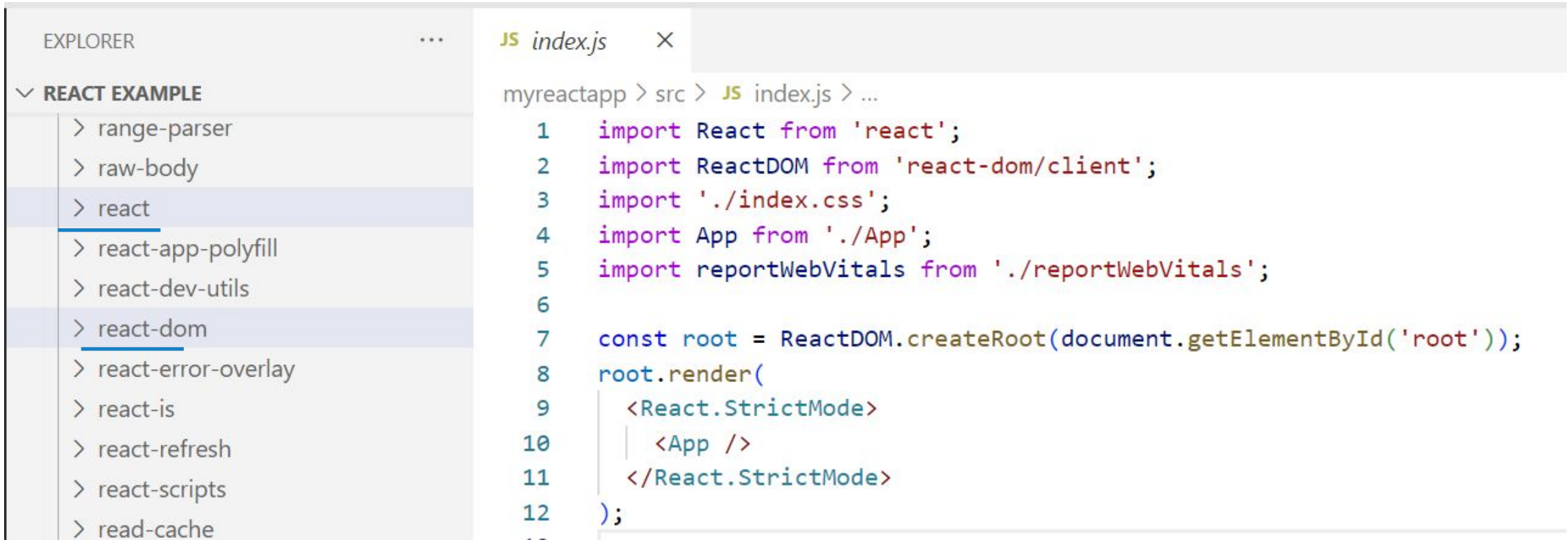
- App.test.js and setupTest.js are used for testing purpose
- reportWebVitals.js is used to generate reports

Index.js

Index.js is the main file that is executed first



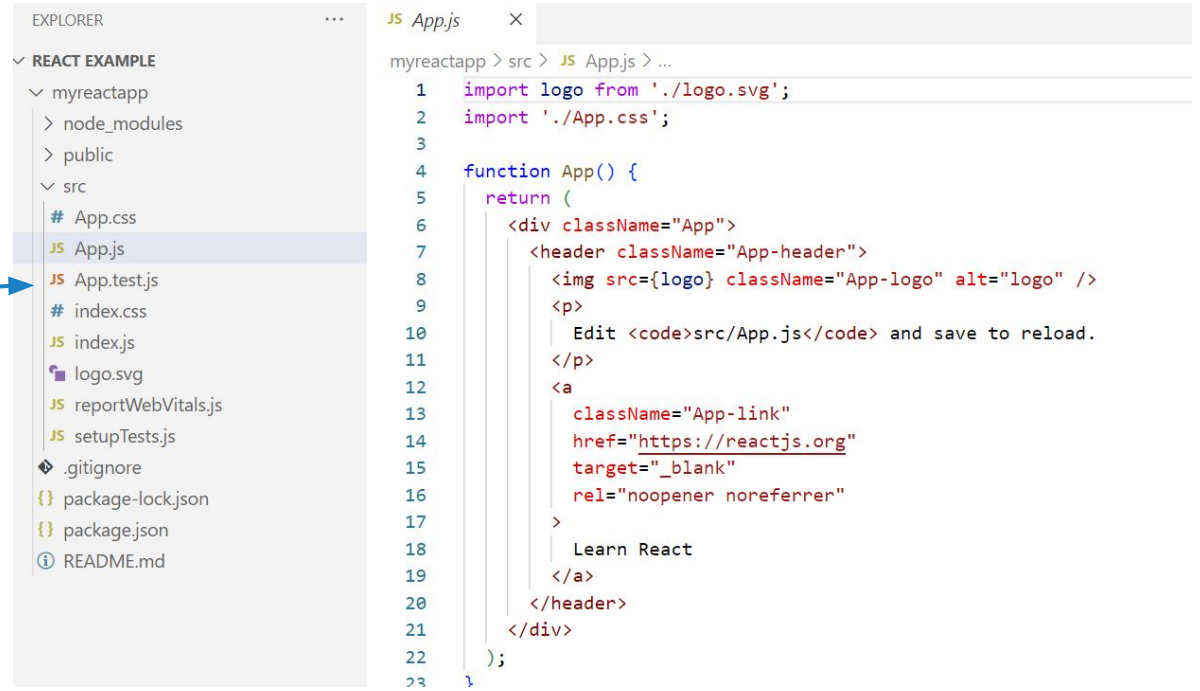
Importing the code from React (inside Node _Modules)



Here index.css contains universal components for the website
.css files and images can be imported

Code Walkthrough Index.js

```
import App from './App';
import reportWebVitals from './reportWebVitals';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```



Here **ReactDOM is the Object** imported from react-dom and render is the method which specifies the component that needs to be added in the webpage.

App: Component is created in App.js and imported in Index.js

<App /> is JSX // this component has to be rendered inside root element (index.html)

JSX stands for JavaScript XML.

JSX allows us to write HTML in React.

JSX makes it easier to write and add HTML in React.

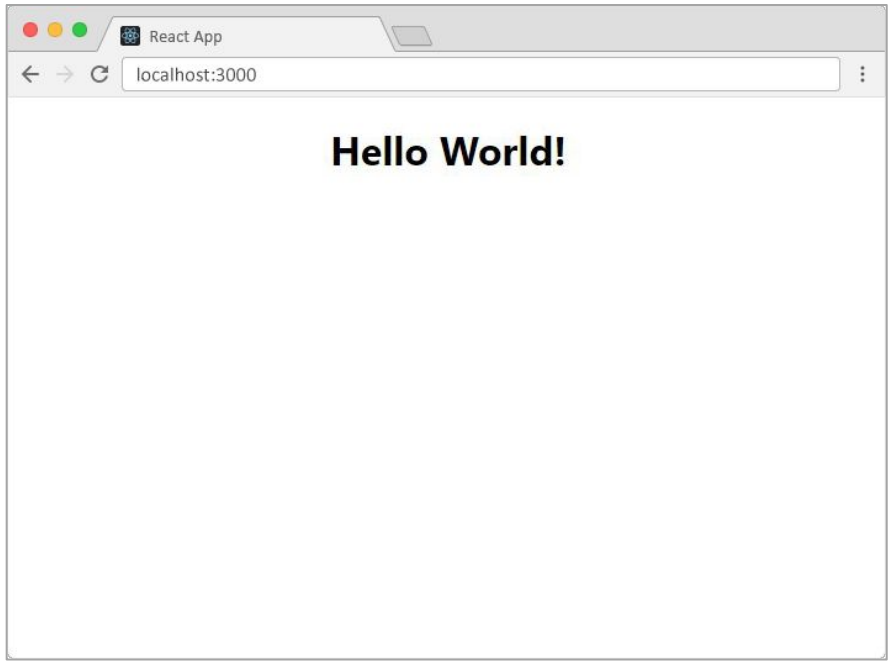
Getting Started

src/index.js

```
import React from 'react'
import ReactDOM from 'react-dom'
import './index.css'

class App extends React.Component {
  render() {
    return (
      <div className="App">
        <h1>Hello World</h1>
      </div>
    )
  }
}

ReactDOM.render(<App />, document.getElementById('root'))
```

public/index.html

```
<html>
<head>
    <title>React App</title>
</head>
<body>
    <div id="root"></div>
</body>
</html>
```



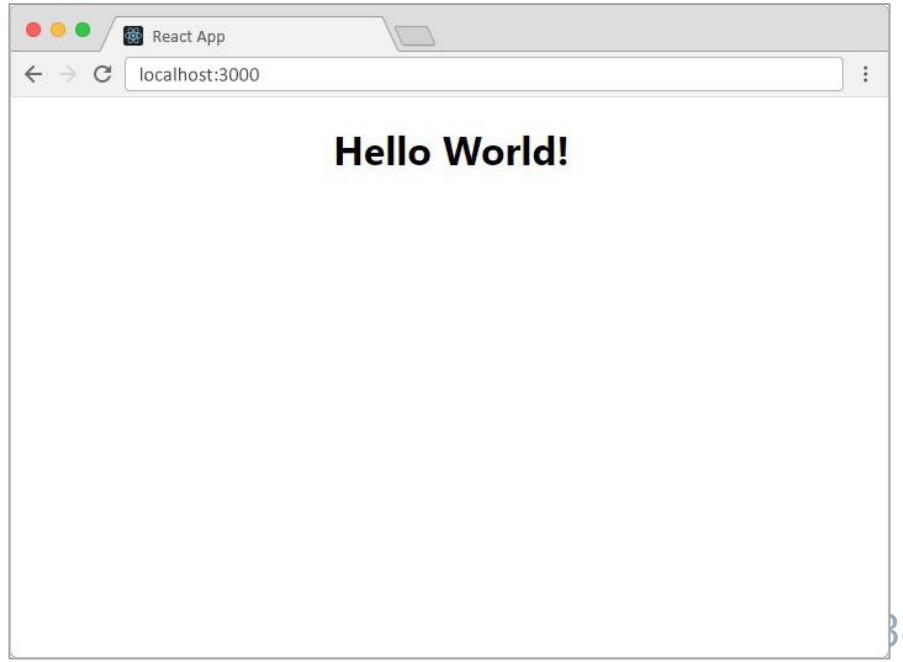
Getting Started

src/index.js (***React 18***)

```
import React from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'

class App extends React.Component {
  render() {
    return (
      <div className="App">
        <h1>Hello World</h1>
      </div>
    )
  }
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

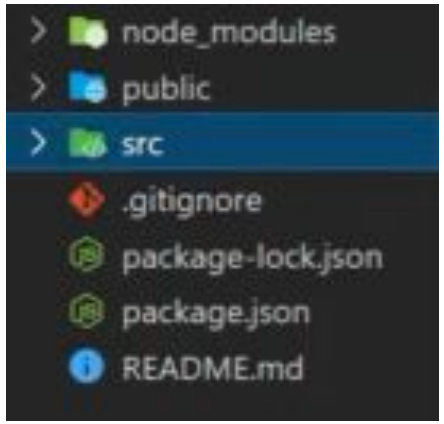
public/index.html

```
<html>
<head>
    <title>React App</title>
</head>
<body>
    <div id="root"></div>
</body>
</html>
```



Creating a project from scratch

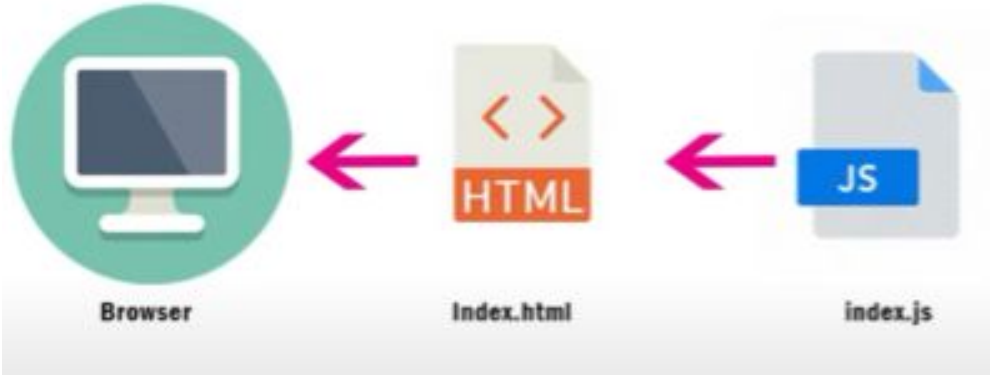
For creating a new project navigate to src folder and delete all the files.



If all files are deleted , then server shows the following error

- It signifies that Index.js is very important
- Public folder has index.html

Create a file Index.js inside src folder



Getting Started Hello React app

src/index.js

index.js

myreactapp > src > index.js

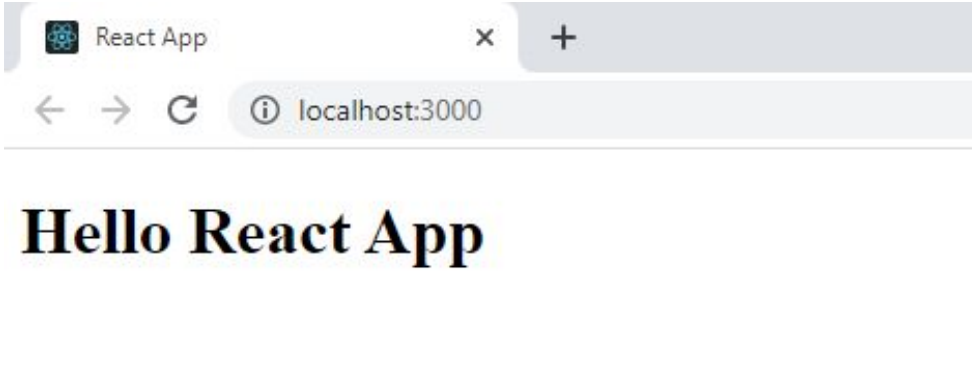
```
1  import React from 'react'
2  import ReactDOM from 'react-dom'
3
4  ReactDOM.render(<h1>Hello React App </h1>, document.getElementById('root'));
5
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
Local:            http://localhost:3000
On Your Network:  http://172.16.208.83:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

public/index.html

```
<html>
 head>
    <title>React App</title>
/head>
body>
    <div id="root"></div>
/body>
/html>
```



Attaching universal components using CSS

JS index.js # index.css ×

myreactapp > src > # index.css > ...

```
1   body {
2   background-color: azure;
3   }
```

public/index.html

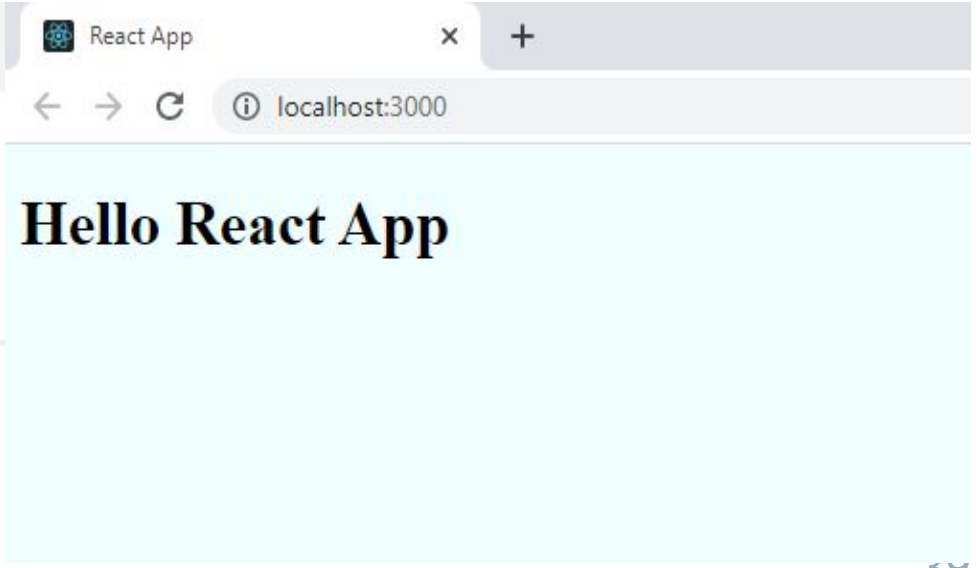
```
<html>
<head>
    <title>React App</title>
</head>
<body>
    <div id="root"></div>
</body>
</html>
```

src/index.js

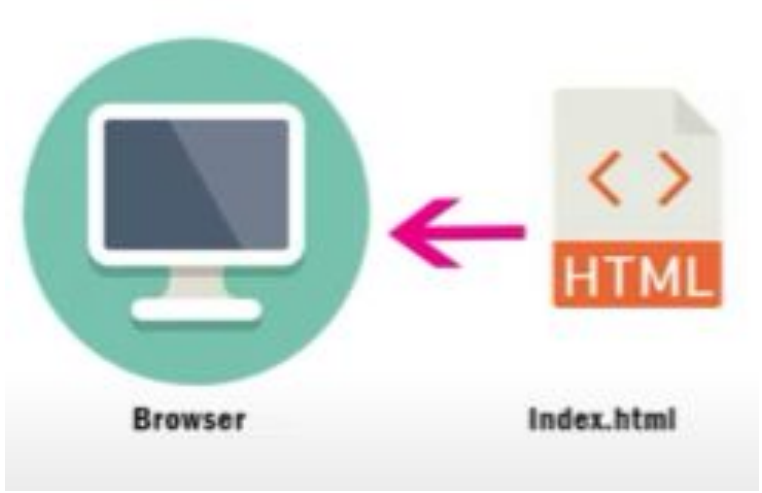
JS index.js × # index.css

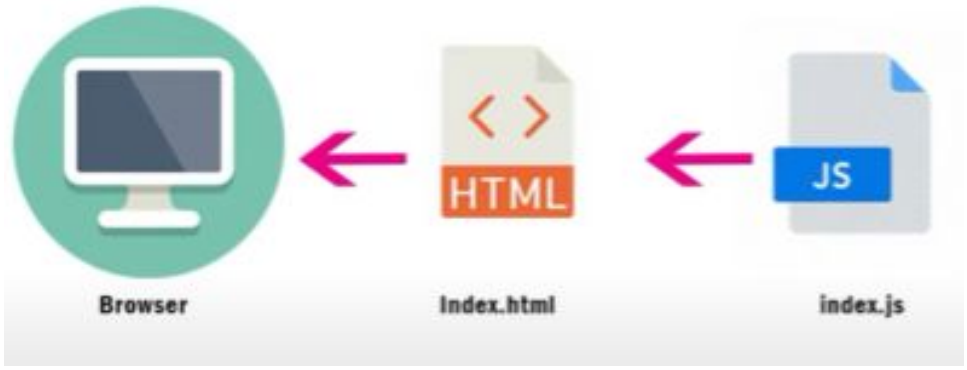
myreactapp > src > JS index.js

```
1   import React from 'react'
2   import ReactDOM from 'react-dom'
3   import './index.css'
4   ReactDOM.render(<h1>Hello React App </h1>, document.getElementById('root'));
```

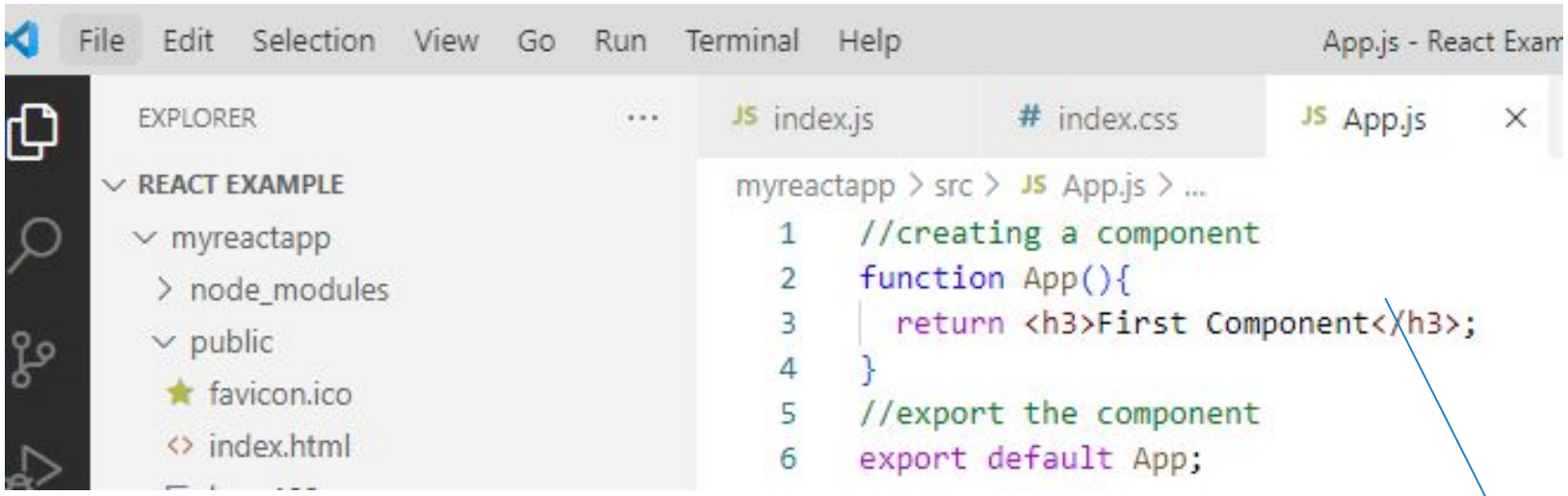


Creating a project and adding the components

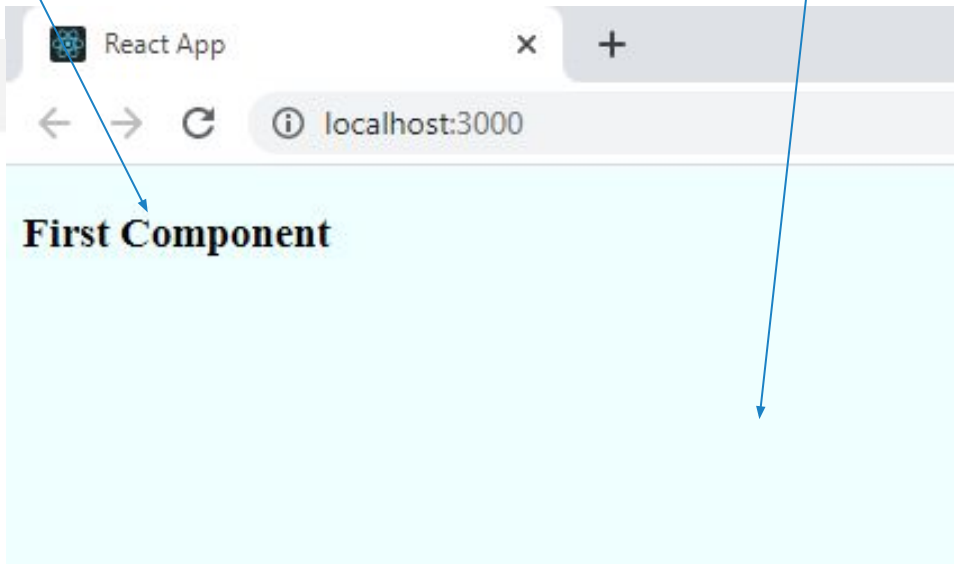




Adding components App.js



Index.css universal styling



Adding multiple components App.js

myreactapp > src > JS App.js > ...

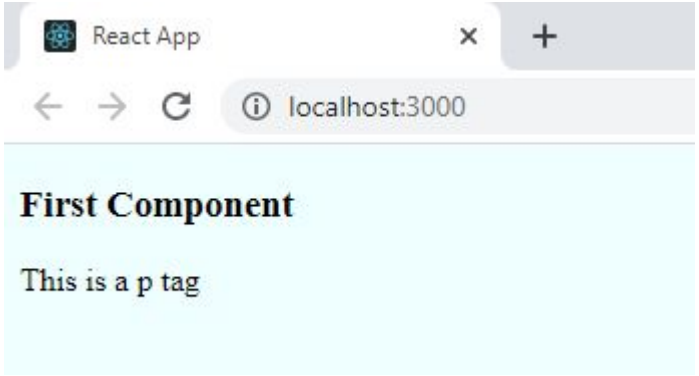
```
1   //creating a component
2   function App(){
3     return <h3>First Component</h3><p>This is a p tag</p>;
4   }
5   //export the component
6   export default App;
7
```

The above code shows error, since we can only return single value. We can pass only one tag
Add a wrapper using div .

JS index.js # index.css JS App.js

myreactapp > src > JS App.js > ...

```
1   //creating a component
2   function App(){
3     return (
4     <div>
5     <h3>First Component</h3>
6     <p>This is a p tag</p>
7     </div>
8     );
9   }
10  //export the component
11  export default App;
```



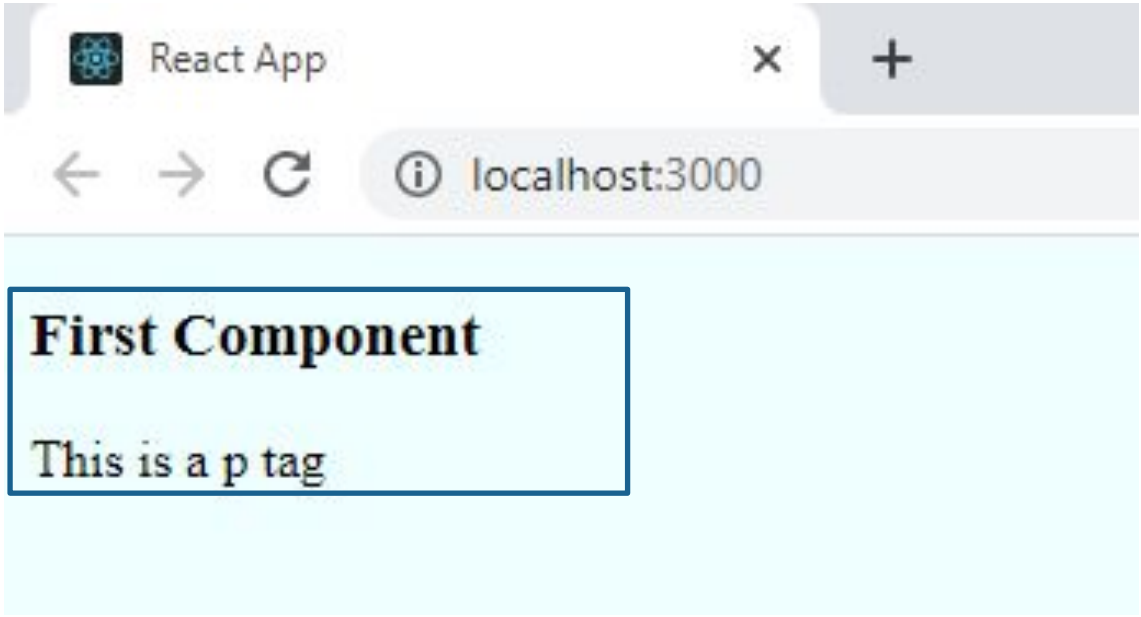
Styling universal and different components

- Every Component can be styled
- Main .css file is : Index.css where all universal components are placed
- Every component that can be reused is styled in a different .css file . For example, in the above project Div component can be styled using App.css

index.js | index.css | App.js

myreactapp > src > App.js > ...

```
1   //creating a component
2   function App(){
3     return (
4     <div>
5     <h3>First Component</h3>
6     <p>This is a p tag</p>
7     </div>
8     );
9   }
10  //export the component
11  export default App;
```



Styling universal and different components

JS index.js × # index.css JS App.js # App.css ●

myreactapp > src > # App.css > ...

```
1   .app-container{
2     background-color: coral;
3   }
```

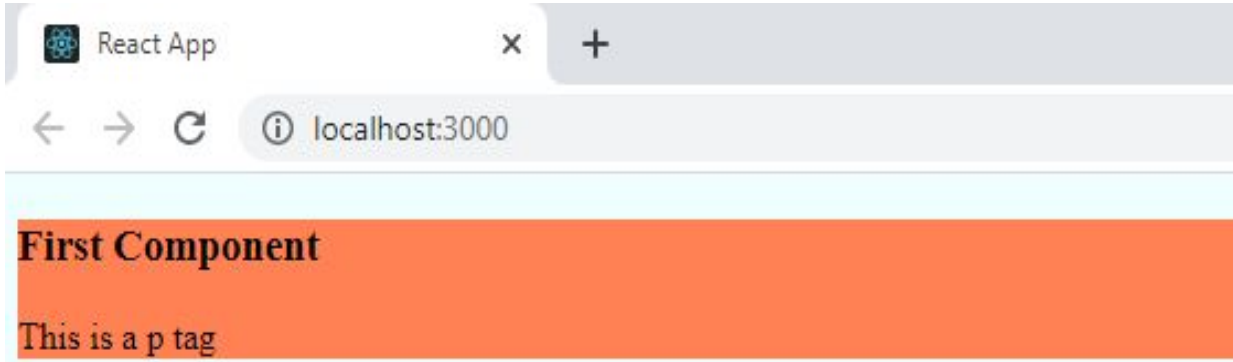
Create a class for instance, “app-container”

JS index.js # index.css JS App.js × # App.css

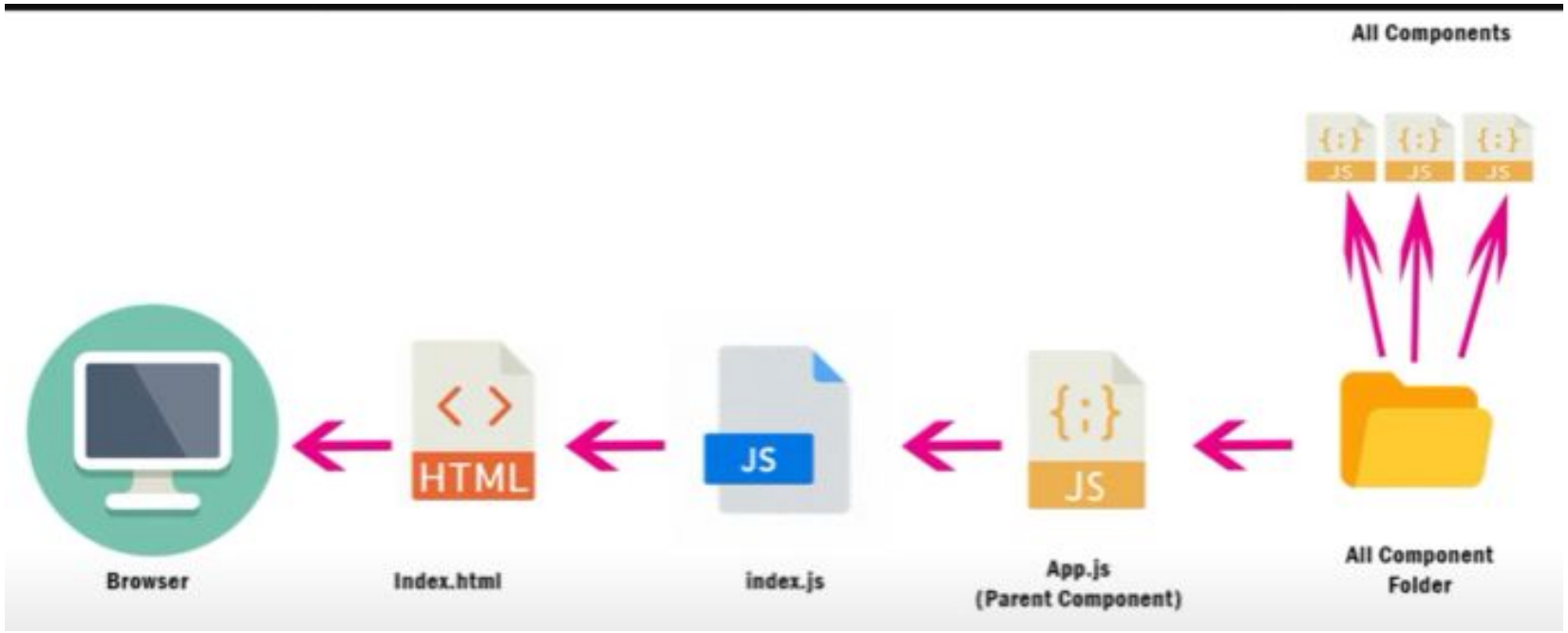
myreactapp > src > JS App.js > ...

```
1   import './App.css';
2
3   //creating a component
4   function App(){
5     return (
6     <div className="app-container">
7     <h3>First Component</h3>
8     <p>This is a p tag</p>
9     </div>
10    );
11  }
12  //export the component
13  export default App;
```

Import App.cs in App.js, since class is a reserved word , in JavaScript use keyword **className** and pass name of class created in App.cs

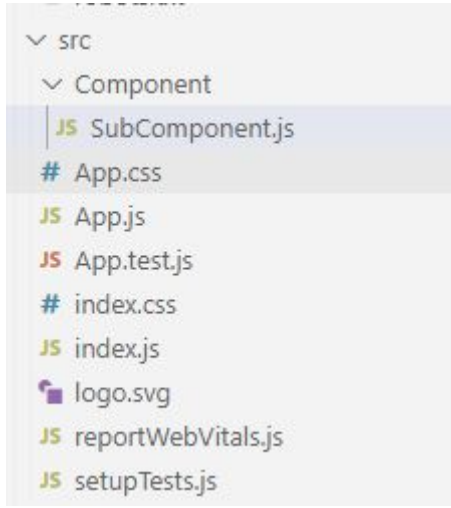


Revisiting all the files



Adding sub components (.js)

- Create a separate folder Component to add several components according to app. For instance, create SubComponent.js and add the following code



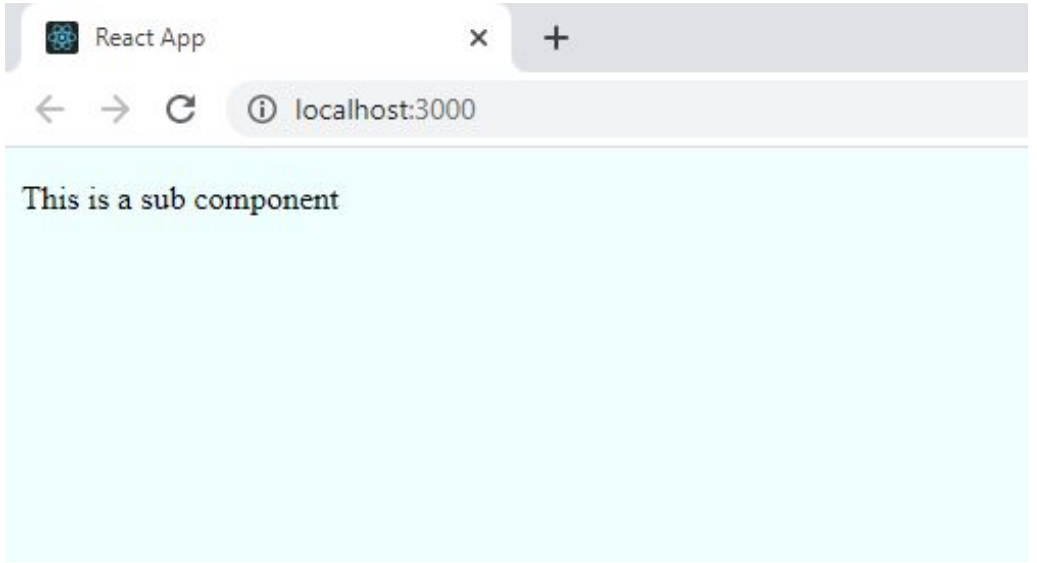
myreactapp > src > Component > JS SubComponent.js > default

```
function SubComponent(){
    return (
        <p>This is a sub component</p>
    );
}
export default SubComponent;
```

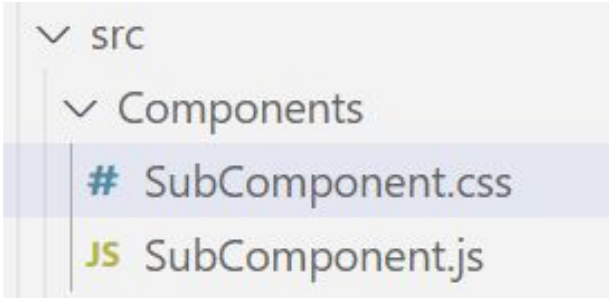
- In App.js add the above created component

myreactapp > src > JS App.js > ...

```
import './App.css';
import SubComponent from './Component/SubComponent';
//creating a component
function App(){
  return (
    // Here we will pass subComponent since it is Parent
    <SubComponent />
  );
}
//export the component
export default App;
```



Adding css file in subcomponent



Create a new file SubComponent.css

JS index.js | JS App.js | # App.css | JS SubComponent.js | # SubComponent

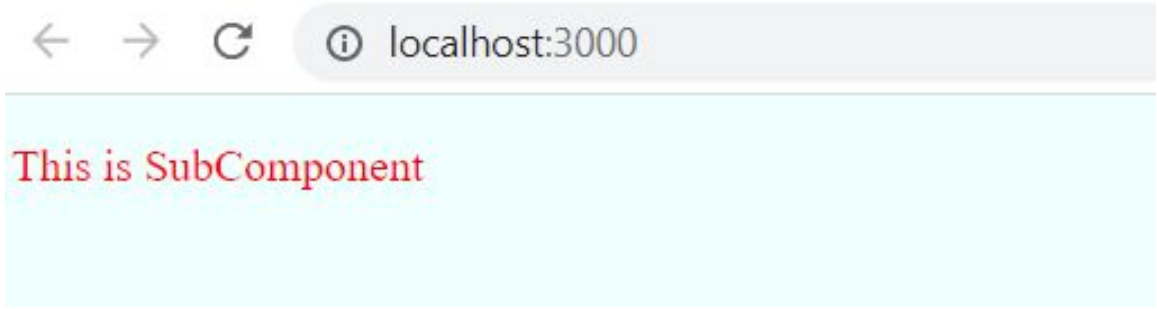
myreactapp > src > Components > # SubComponent.css > .redColor

```
1   .redColor{
2       color: red;
3       font-size: "50px";
4   }
```

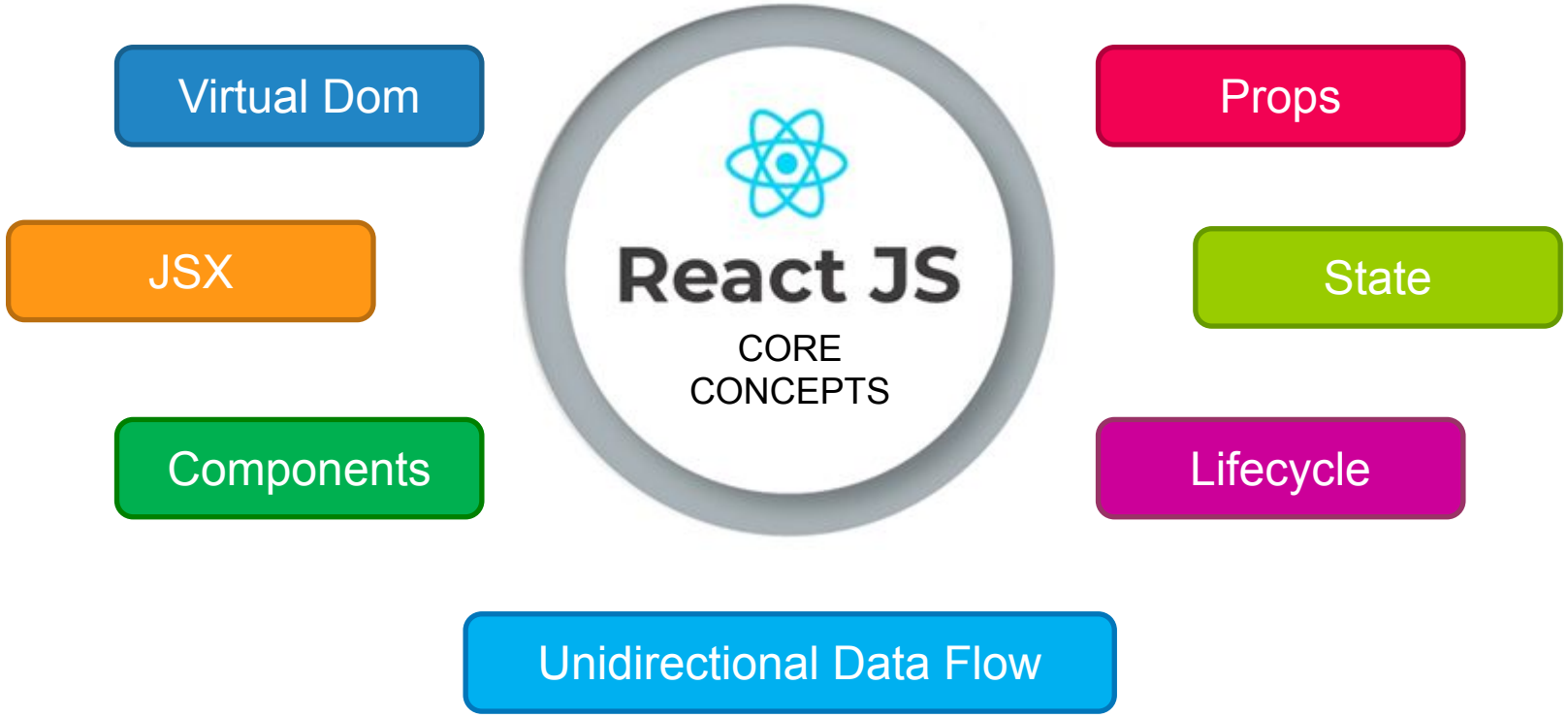
JS index.js | JS App.js | # App.css | JS SubComponent.js | # SubComponent.css

myreactapp > src > Components > JS SubComponent.js > default

```
1   import './SubComponent.css'
2   function SubComponent ()
3   {
4       return (<p className="redColor"> This is SubComponent </p>)
5   }
6
7   export default SubComponent;
```



Core Concepts



Document Object Model (DOM)

- The **Document Object Model** (**DOM**) is a cross-platform and language-independent interface that treats an XML or HTML document as a tree structure wherein each node is an object representing a part of the document.[1]
- When a web page gets loaded, the browser creates a DOM of the page, which is constructed as a **Tree of Objects** from all elements on that web page.
- Using DOM interface gives JS all the power it needs to create **Dynamic HTML**.

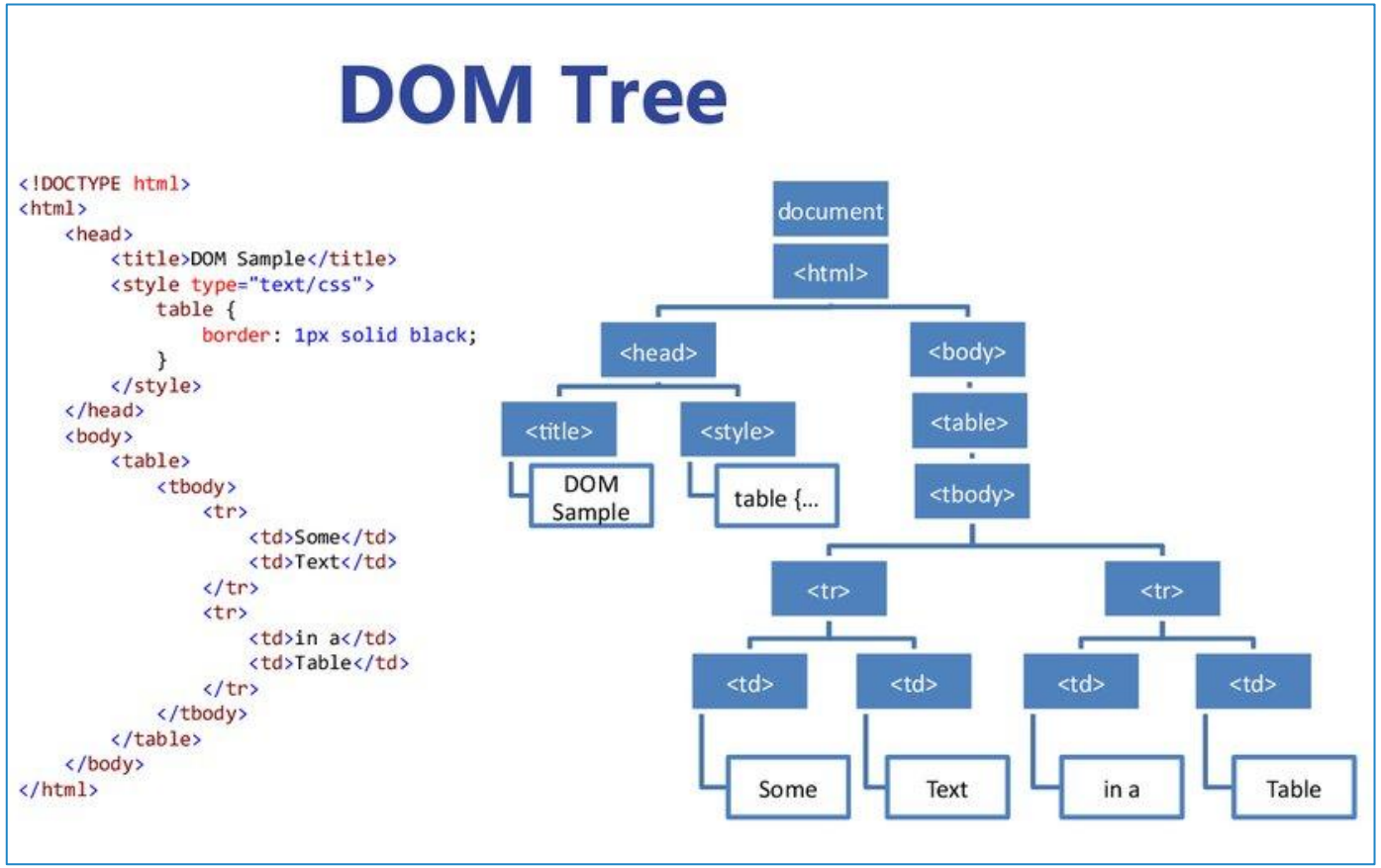


Image Courtesy: https://en.ppt-online.org/83335
Text Courtesy: [1] https://en.wikipedia.org/wiki/Document_Object_Model

Virtual DOM

What is a virtual DOM?

- A virtual DOM object is a *representation* of an actual DOM object, like a lightweight copy.
- Virtual DOM object has the same properties as a real DOM object, but unlike the real object it cannot directly change what's being displayed on the screen.
- Manipulating the regular DOM is slow. Manipulating the virtual DOM is much faster.

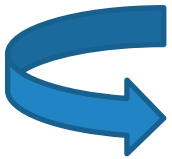
How it helps?

Whenever something changes on the screen

1. A snapshot of the virtual DOM is taken and stored.
2. The entire virtual DOM then gets updated.
3. The updated virtual DOM is compared with the pre-updated snapshot. It is figured out as to which objects have changed.
4. Thereafter, only the changed objects get updated in the real DOM.
5. Changes on the real DOM cause the web page to change.

JavaScript XML (JSX)

- JSX allows us to write HTML in React
- Under the hood, it runs *createElement()*, which takes the tag, object containing the properties, and children of the component and renders the same information.



```
const heading = <h1 className="site-heading">Hello, React</h1>

const heading = React.createElement('h1', {className: 'site-heading'}, 'Hello, React!')
```

A few key points to remember when using JSX

- *className* is used instead of *class* for adding CSS classes, as *class* is a reserved keyword in JS
- Properties and methods in JSX are camelCase (*onclick* will become *onClick)*
- Self-closing tags must end in a slash - e.g. <img />

JavaScript XML (JSX)

- To write HTML on multiple lines, put the HTML inside parentheses
- Multi-line HTML code must be wrapped in ONE top level parent element

```
const myDiv = (
 <div>
    <h1>REACT IN THE REAL WORLD</h1>
    <p>React has become arguably the most popular JavaScript framework
       currently in use </p>
  </div>
);
```

Expressions in JSX

- JavaScript expressions can be embedded inside JSX using curly braces
- The expression can be a variable, a function, a property, or any valid JS expression

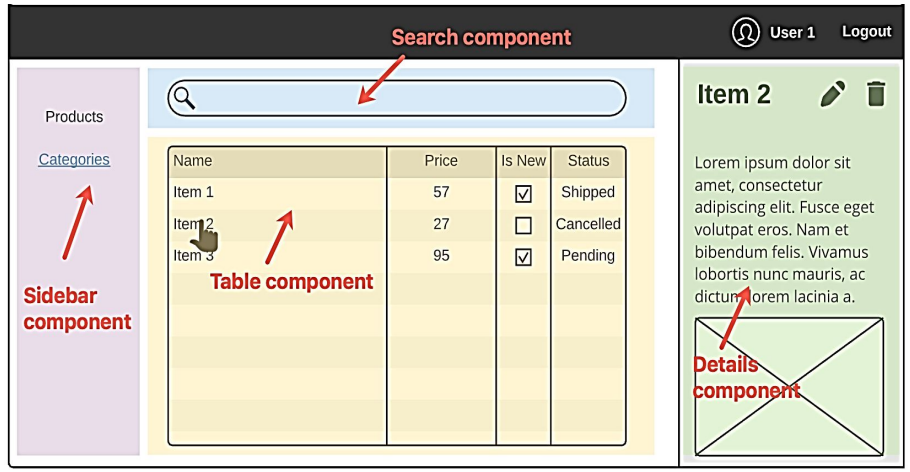
```
const txt1 = <h1>React is {5 * 2} times better with JSX</h1>;

const name = 'Anu'
const txt1 = <h1>Hello, {name}</h1>

const myId = 'test'
const element = <h1 id={myId}>Hello, world!</h1>
```

Components

- A component is one isolated piece of user interface
- Components are independent and reusable bits of codes that collectively form the UI
- Components are like JS functions that return HTML elements
- Component's name must start with an upper case letter
- Components are of two types,

  Stateful components

  Stateless components
- React apps have multiple components, and each component can be placed in a separate file
- They serve the same purpose as JavaScript functions, but work in isolation and return HTML via a render() function.



Components come in two types, Class components and Function components,

Class component

- When creating a React component, the component's name must start with an upper case letter.
- The component has to include the extends React.Component statement, this statement creates an inheritance to React.Component, and gives your component access to React.Component's functions.
- The component also requires a render() method, this method returns HTML.

p.js | JS MyClass.js × | # index.css

ctapp > src > JS MyClass.js > ...

```
import React from 'react'

class MyClass extends React.Component{
    render() {
        return(
            <div>
                <h1> This is a Class Component </h1>
            </div>
        ) }}

export default MyClass
```

JS App.js × | # index.css

myreactapp > src > JS App.js > ...

```
1   import MyClass from './MyClass'
2   function App()
3   {
4       return(<MyClass/>);
5   }
```

localhost:3000

This is a Class Component

Props

- Props are arguments passed to React components, by using HTML attributes
- They are like function arguments in JavaScript. They are a type of object where the value of attributes of a tag is stored.
- They store the value of a tag's attributes and work similar to the HTML attributes
- Props are read-only
- All React components must act like pure functions with respect to their props
- Every parent component can pass some information to its child components by giving them props.
- Props are the only argument to your component
- React component functions accept a single argument, a props object

Component

myreactapp > src > JS App.js > ...

```
1   import React from 'react'
2   import ReactDOM from 'react-dom'
3
4   function App () {
5       return(
6           <div>
7               <h2>Printing Details</h2>
8               <h3>John</h3>
9               <h3>Doe</h3>
10              <h3>58</h3>
11
12          </div>
13      )
14  }
15  export default App;
```

JS index.js × JS App.js ●

myreactapp > src > JS index.js

```
1   import React from 'react'
2   import ReactDOM  from 'react-dom'
3   import './index.css'
4   import App from './App'
5
6   ReactDOM.render(
7     <App/>,
8     document.getElementById('root')
9   );
```

localhost:3000

Printing Details

John

Doe

58

Component

```
import React from 'react'
import ReactDOM from 'react-dom'

function App () {
//create variable
let fname="Jane"
    return(
        <div>
            <h2>Printing Details</h2>
            <h3>{fname}</h3>
            <h3>Doe</h3>
            <h3>58</h3>

        </div>
    )
}
export default App;
```

index.js | App.js

myreactapp > src > index.js

```
1  import React from 'react'
2  import ReactDOM  from 'react-dom'
3  import './index.css'
4  import App from './App'
5
6  ReactDOM.render(
7    <App/>,
8    document.getElementById('root')
9  );
```

localhost:3000

Printing Details

Jane

Doe

58

Passing data inside component using props

dex.js × JS App.js

eactapp > src > JS index.js > ...

```
import React from 'react'
import ReactDOM  from 'react-dom'
import './index.css'
import App from './App'

//creating variables
let fname="Joseph"
let lname="Doe"
let age=45
ReactDOM.render(
 // Here App component was used
 // Thus, the data will be shared from here
  <App
  firstName={fname}
  lastName={lname}
  Age={age}
  />,
  document.getElementById('root')
);
```

JS index.js JS App.js ×

myreactapp > src > JS App.js > ...

```
1   import React from 'react'
2   import ReactDOM from 'react-dom'
3
4   function App (props) {
5   //create variable
6       return(
7           <div>
8               <h2>Printing Details</h2>
9               <h3>{props.firstName}</h3>
10              <h3>{props.lastName}</h3>
11              <h3>{props.Age}</h3>
12
13          </div>
14      )
15  }
16  export default App;
```

localhost:3000

Printing Details

Joseph

Doe

45

Props

x.js ● JS App.js ●

tapp > src > JS index.js > ...

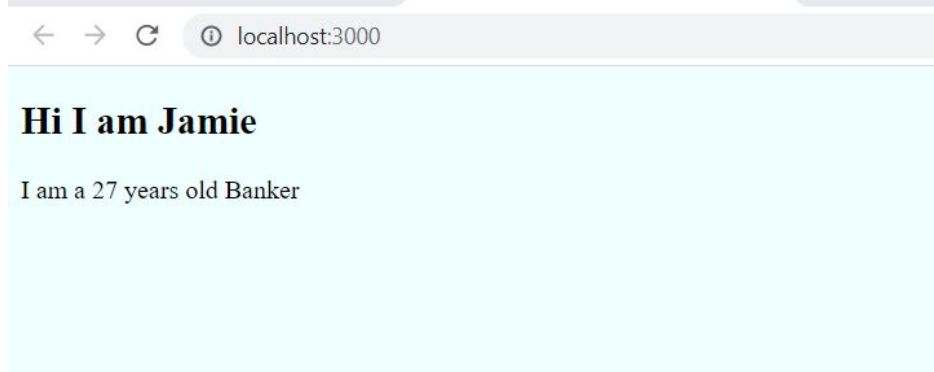
```
import React from 'react'
import ReactDOM  from 'react-dom'
import './index.css'
import Hello from './App'

let name_1="Jamie";

ReactDOM.render(
  <Hello name={name_1} age={27} occupation="Banker"/>,
  document.getElementById('root')
);
```

this.props is the object which contains the Attributes which we have passed to the parent component

js JS App.js ●

pp > src > JS App.js > ...

```
import React from 'react'
import ReactDOM from 'react-dom'

class Hello extends React.Component{
  render(){
  return(
    <div>
      <h2>Hi I am {this.props.name} </h2>
      <p>I am a {this.props.age} years old {this.props.occupation} </
    </div>
  );
}
}

export default Hello
```



Props

dex.js | JS App.js

eactapp > src > JS App.js > ...

```
import React from 'react'
import ReactDOM from 'react-dom'

class Age extends React.Component{
render(){
 let bornyear=this.props.details.year;
 return(
  <h2>Hey {this.props.details.name}, you are born in the year {bornyear}. </h2>
 );}}

class Person extends React.Component{
  render(){
    const personinfo= {name:"Chris", year:1976};
    return(
      <div>
        <Age details={personinfo} />
      </div>);}}

export default Person
```

JS index.js | JS App.js

myreactapp > src > JS index.js

```
1   import React from 'react'
2   import ReactDOM  from 'react-dom'
3   import './index.css'
4   import Person from './App'
5
6   ReactDOM.render(
7     <Person />,
8     document.getElementById('root')
9   );
```

localhost:3000

Hey Chris, you are born in the year 1976.

State

- A State is an object inside the constructor method of a class which is a must in the stateful components. It is used for internal communication inside a component. It allows you to create components that are interactive and reusable. It is mutable and can only be changed by using setState() method.
- Stateful components are those components which have a state. The state gets initialized in the constructor. It stores information about the component's state change in memory. It may get changed depending upon the action of the component or child components.
  - Props and state are rendered like {this.props.name} and {this.state.name} respectively.
- Stateless components are those components which don't have any state at all, which means you can't use this.setState inside these components. It is like a normal function with no render method.
  - In stateless components, the props are displayed like {props.name}

Stateless Components

- Stateless component can be created using function or class
- They hold no state of their own
- They may receive props from other components

```
function Welcome(props) {
    return <h1>Hello, {props.name}</h1>;
}
```

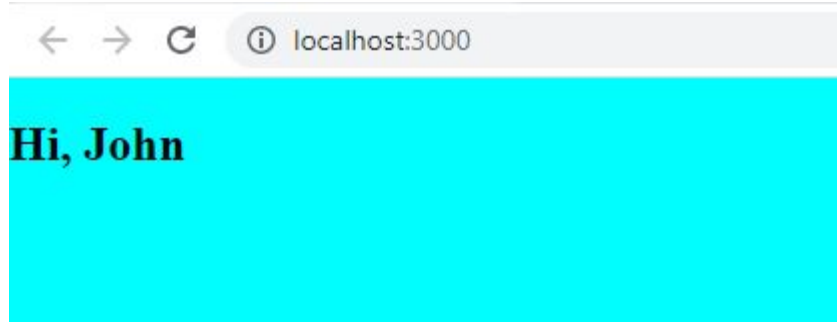
```
class Hello extends React.Component {
  render() {
    return <h1>Hello World!</h1>;
  }
}
```

Class Components in Files

App.js | index.js

myreactapp > src > App.js > ...

```
1  import React from "react";
2  import  ReactDOM  from "react-dom";
3  class Hello extends React.Component{
4      render()
5      {
6          return <h2> Hi, {this.props.name} </h2>
7      }
8  }
9  export default Hello;
10
```



App.js | index.js

myreactapp > src > index.js

```
1  import React from 'react';
2  import ReactDOM from 'react-dom';
3  import './index.css';
4  import Hello from './App';
5  // Here we are passing the component using HTML tag
6  //name of the component will always start from Capital
7  ReactDOM.render(<Hello name="John" />, document.getElementById('root'));
8
```

Components in Files

```
// app.js
import React from 'react';
import ReactDOM from 'react-dom';

class Hello extends React.Component {
    constructor(props) {
        super(props);
    }

    render() {
        return <h2>Hi, {this.props.name}</h2>;
    }
}

export default Hello;
```

If your component has a constructor function, the props should always be passed via the super() method.

super() is used to call the parent constructor, whereas, super(props) would pass the props to the parent constructor.

```
// index.js
import React from 'react'
import ReactDOM from 'react-dom'
import './index.css'
import Hello from './app.js';

ReactDOM.render(<Hello name="Jane"/>, document.getElementById('root'))
```

Nested Components

```
const TableHeader = () => {
  return (
    <thead>
      <tr>
        <th>Brand</th>
        <th>Headquarters</th>
      </tr>
    </thead>
  )
}

const TableBody = () => {
  return (
    <tbody>
      <tr>
        <td>Tata</td>
        <td>India</td>
      </tr>
      <tr>
        <td>Toyota</td>
        <td>Japan</td>
      </tr>
    </tbody>
  )
}
```

```
class Table extends Component {
  render() {
    return (
      <table>
        <TableHeader />
        <TableBody />
      </table>
    )
  }
}
```

Stateful Components

- A stateful component is always a class component
- It is created by extending the React.Component class
- It has a separate render() method for returning JSX on the screen
- It may hold and manage its own state
  - component properties are kept in an object called state
- Its behavior can be made reactive to its state

```
class Car extends React.Component {
  constructor() {
    super();
    this.state = {
      brand: "Ford",
      model: "Mustang",
      color: "red",
      year: 1964
    };
  }
  render() {
    return (
      <div>
        <p> I have a {this.state.color}
            {this.state.brand}
            {this.state.model}
            from {this.state.year}.
        </p>
      </div>
    );
  }
}
```

Stateful components

App.js index.js

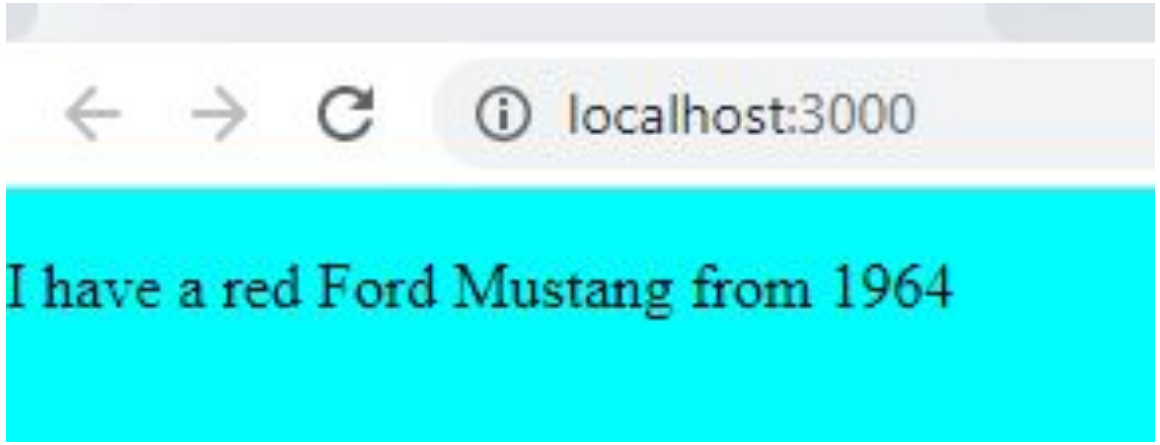
myreactapp > src > App.js > ...

```
1   import React from 'react'
2   import ReactDOM from 'react-dom'
3   class Car extends React.Component{
4       constructor(){
5           super();
6           this.state={
7               brand:"Ford ",
8               model:"Mustang ",
9               color:"red ",
10              year:1964
11          }; }
12      render(){
13      return(
14          <div>
15              <p> I have a {this.state.color}
16              {this.state.brand}
17              {this.state.model}
18              from {this.state.year}
19              </p>
20          </div>);}}
21  export default Car
```

App.js index.js

myreactapp > src > index.js

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import './index.css';
4   import Car from './App';
5   // Here we are passing the component using HTML tag
6   //name of the component will always start from Capital
7   ReactDOM.render(<Car/>, document.getElementById('root'));
8
```



State

- The state is a built-in React object that is used to store data/ information about the component
- The component re-renders when the state object changes
- The state object is initialized in the constructor with as many properties as desired
- *this.state.propertyname - used to* access the state object properties anywhere in the component

index.js App.js

myreactapp > src > App.js > ...

```
1   import React from 'react'
2   import ReactDOM from 'react-dom'
3
4   class Person extends React.Component{
5     constructor(props)
6     {
7       super(props);
8       this.state={name:this.props.name, occupation:"Chef"};
9     }
10    render(){
11      return(
12        <div>
13          <h2>Hi I am {this.state.name}
14          <br/>I am a {this.state.occupation} </h2>
15        </div>);}}
16
```

index.js App.js

myreactapp > src > index.js

```
1   import React from 'react'
2   import ReactDOM  from 'react-dom'
3   import './index.css'
4   import Person from './App'
5
6   ReactDOM.render(
7     <Person name="Jamie"/>,
8     document.getElementById('root')
9   );
```

localhost:3000

Hi I am Jamie
I am a Chef

State

index.js | App.js

myreactapp > src > App.js > ...

```
1   import React from 'react'
2   import ReactDOM from 'react-dom'
3   //setState()
4   class Person extends React.Component{
5     constructor(props){
6       super(props);
7       this.state={name:this.props.name, occupation:"Chef"};}
8
9     //setState()
10    changeDetails = () => {
11    this.setState({occupation:"Blogger", name:"Jane"});
12    }
13     render(){
14       return(
15         <div>
16           <h2>Hi I am {this.state.name}
17           <br/>I am a {this.state.occupation} </h2>
18           <button type="button" onClick={this.changeDetails}> Change Details</button>
19         </div>);}}
20
21  export default Person
```

this.setState() method is used to change the value of a property in the state object

index.js | App.js

myreactapp > src > index.js

```
1   import React from 'react'
2   import ReactDOM  from 'react-dom'
3   import './index.css'
4   import Person from './App'
5
6   ReactDOM.render(
7     <Person name="Jamie"/>,
8     document.getElementById('root')
9   );
```

Declarative code in JSX

Declarative programming **focuses on the WHAT rather than the HOW**.

It is driven by describing the end result, rather than specifying the step by step process of getting to the end result.

```
function addCourseNameToBody() {
    const bodyTag = document.querySelector('body')
    const divTag = document.createElement('div')
    let h1Tag = document.createElement('h1')
    h1Tag.innerText = "Learning React"
    divTag.append(h1Tag)
    bodyTag.append(divTag)
}
```

```
class Course extends Component {
    render() {
        return(
            <div>
            <h1>{this.props.name}</h1>
        </div>
        );
    }
}
```

Applying style in JSX

```
class Styles extends React.Component {
    h2Style = {
        fontSize: 40,
        textDecoration: 'underline'
    };

    render() {
        return (
            <div>
                <h2 style={this.h2Style}>React JS</h2>

                <p style={ {fontStyle: 'italic', color:'blue'} }>
                    React is a JavaScript library that aims to
                    simplify development of visual interfaces.
                </p>

                <button className="btn btn-success">Get Started</button>
            </div>
        );
    }
}
```

As a property

Inline Style

Using Classes

React Lifecycle Methods

Every React component's lifecycle consists of three main phases –

- **MOUNTING**

  (mounted on the DOM)
- **UPDATING**

  (state/ props get updated)
- **UNMOUNTING**

  (unmounted from the DOM)

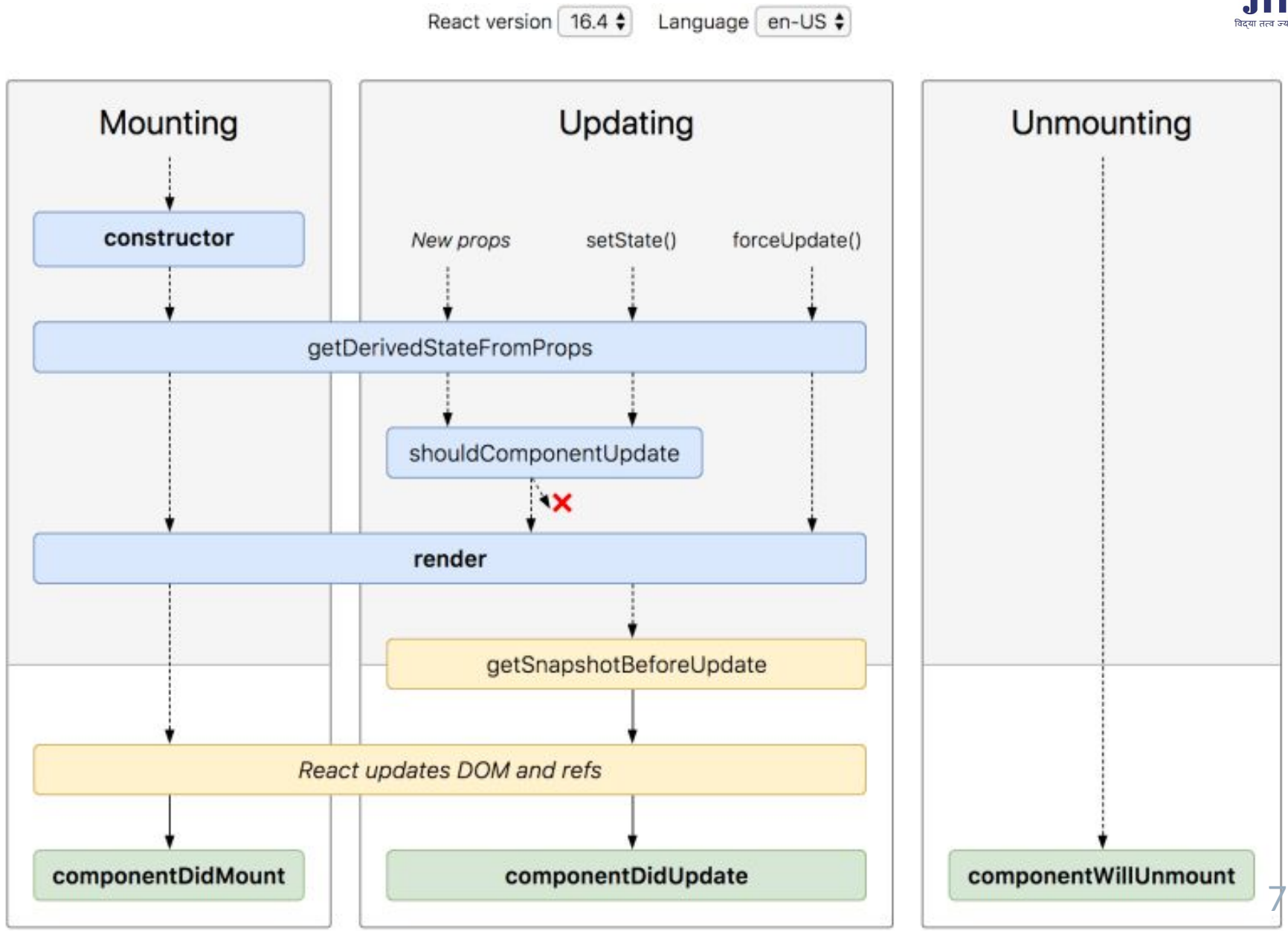


Image Courtesy:
https://projects.wojtekmaj.pl/react-lifecycle-methods-diagram/

React Lifecycle Methods

- **constructor()** – invoked before the component is mounted
  - sets up the initial state and values
  - invoked with the props as arguments
  - first statement should be a call to *super(props)*
  - serves two main purposes Initializing local state by assigning an object to *this.state*
    - Binding event handler methods to an instance
- **render()** - most used lifecycle method
  - is the only required method within a class component
  - handles the rendering of your component to the UI
  - invoked during mounting as well as updating phases
  - should be a pure function
  - does not directly interact with the browser
  - will not be invoked if *shouldComponentUpdate()* returns false

React Lifecycle Methods

- **componentDidMount()** – invoked immediately after a component is mounted
  a good place to initiate API calls, if you need to load data from a remote endpoint
  allows the use of *setState()*

- **componentDidUpdate()** – invoked as soon as an update occurs (*not invoked for the initial render*)
  it updates the DOM (*say in response to state/ props change*)
  can call *setState()* in this lifecycle - but wrap it in a condition that checks for state/ prop changes from previous state
  will not be invoked if *shouldComponentUpdate()* returns false

- **componentWillUnmount()** – invoked just before a component is unmounted and destroyed
  cleanup activities should be done in this method
  should not call setState() in this, as the component will never be re-rendered

React Lifecycle Methods

- **getDerivedStateFromProps()** – first method to be invoked when a component gets updated (*if defined*)
  - is invoked prior calling the render method (*both on initial mount & subsequent updates*)
  - is a static function that does not have access to “this“
  - returns an object to update state in response to prop changes
  - returns null if there is no change to state
- **shouldComponentUpdate()** – it is useful when you don’t want to render your state or prop changes
  - lets React know if a component’s output is not affected by a change in state/ props
  - the default behavior is to re-render on every state change
  - is invoked before rendering when new props or state are being received
  - is not called for the initial rendering or when *forceUpdate()* is used
- **getSnapshotBeforeUpdate()** – invoked before the most recently rendered output is committed (*to the DOM*)
  - aids the component to capture some information from the DOM before it is changes
  - value that it returns is passed on to componentDidUpdate()

React Lifecycle Methods – *Error Boundaries*

- **Error boundaries** - React components that handle JS errors anywhere in their child component tree, log those errors, and display a fallback UI

- The following methods are invoked when an error is thrown by a descendant component

  static getDerivedStateFromError() - It receives the error that was thrown as a parameter and returns a value to update the state of the component.

  Rendering of fallback UI can be handled here.

  componentDidCatch() - It takes in two parameters □

  the error that was thrown, and

  an object containing info about which component threw the error

  It can be used for activities like logging errors, etc.

Unidirectional Data Flow (UDF)

UDF means that data has one, and only one, way to be transferred to other parts of the application, generally from top (parent) to bottom (child) components.

This means that:

- state is passed to the view and child components
- actions are triggered by the view
- actions update the state
- the state change is passed to the view and child components

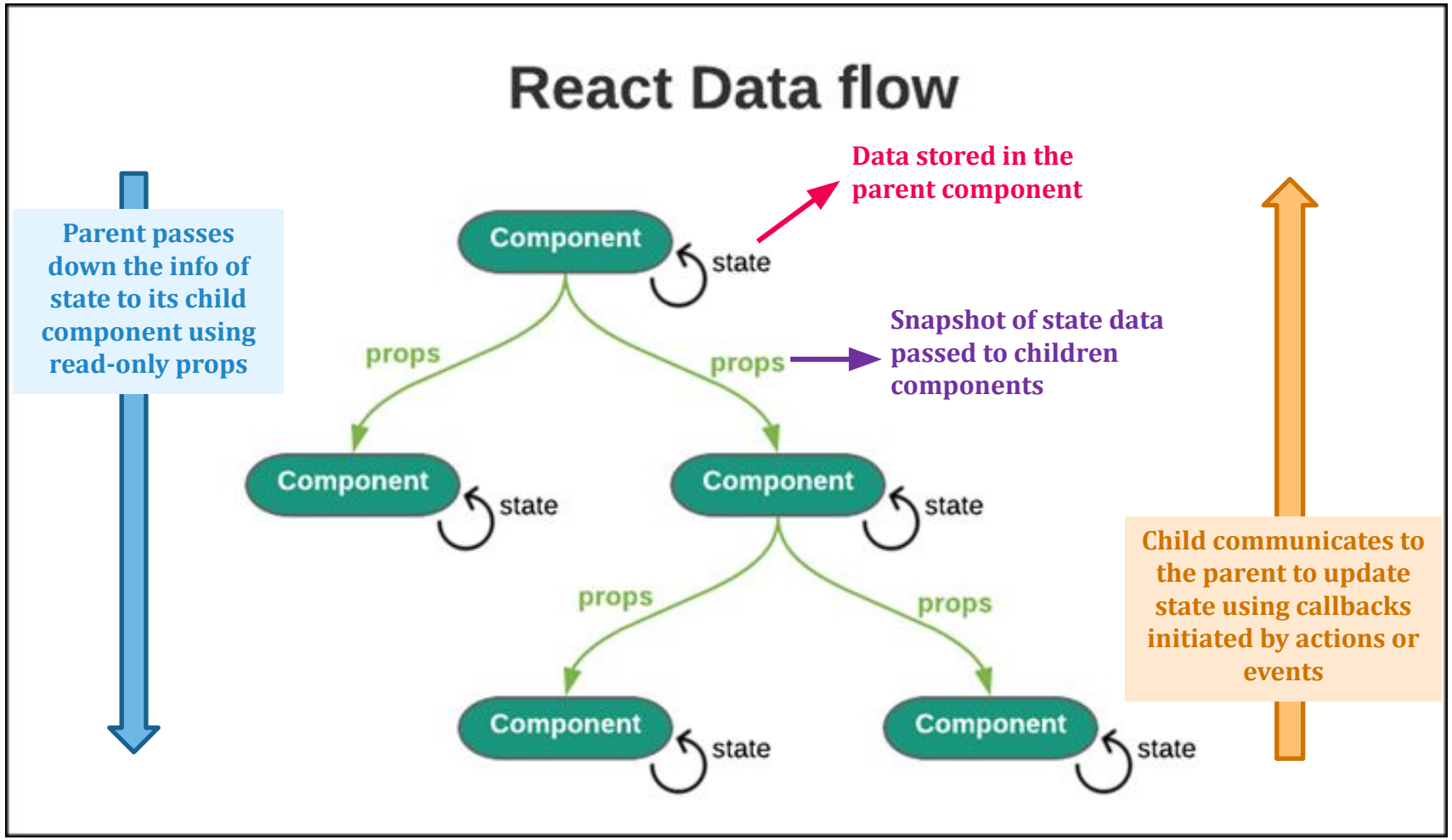


Image Courtesy:
https://medium.com/javascript-in-plain-english/reactjs-training-understanding-react-and-typescript-d01deb2dd127

Some advantages of UDF

- Easier to debug as there is a single source of truth, i.e., the state is maintained at a single location, it can be easily traced what actions are updating the data as the flow is unidirectional.
- You can control which components to re-render on a common state change. You put all those components under one common parent component.
- Easy to trace and fix bugs and errors caused by any bad code
- Leads to a predictable and clean data flow architecture allowing more control over the data.
- Less error prone, as you have more control over the data.

React Events

React has the same events as HTML: click, change, mouseover, keydown etc.

Event Handling

1. **Add event to the element**

   Events are named using camelCase, rather than lowercase

   [onClick instead of onclick]

2. **Include event handler as a method in the component class**

   Event handlers are written inside curly braces

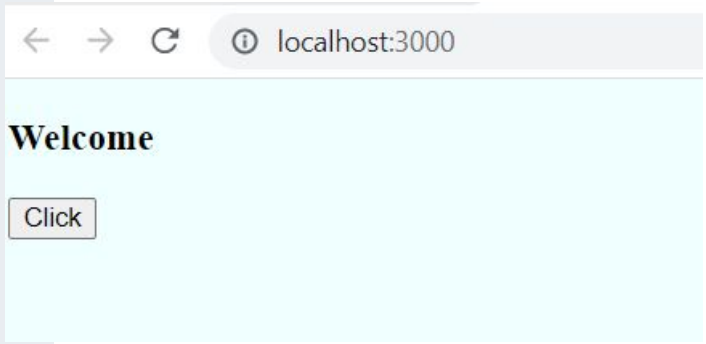
   onClick = {calcSum} [as written in React]

   onClick = "calcSum()" [as written in HTML]

3. **Bind *this*** - using arrow functions bind *this* to the component instance

React Events

```
class EventBind extends Component {
    constructor(props) {
        super(props)
        this.state = {
            message: 'Welcome'
        }
    }

    clickHandler() {
        this.setState({
            message:'Farewell'
        })
    }

    render() {
        return (
            <div>
                <h3>{this.state.message}</h3>
                <button onClick={this.clickHandler}>Click</button>
            </div>
        )
    }
}
```



Nothing will happen on Click Event

Binding Event Handler

Binding Event Handler in Render Method


```
render() {
        return (
            <div>
                <h3>{this.state.message}</h3>
                <button onClick={this.clickHandler.bind(this)}>Click</button>
            </div>
        )
    }
```

Binding Event Handler in the Constructor

```
constructor(props) {
        super(props)
        this.state = {
            message: 'Welcome'
        }
    this.clickHandler = this.clickHandler.bind(this)
    }
```

Binding Event Handler

Binding Event Handler using Arrow Function

```
render() {
        return (
            <div>
                <h3>{this.state.message}</h3>
                <button onClick={() => this.clickHandler()}>Click</button>
            </div>
        )
    }
```

```
clickHandler = () => { this.setState({message:'Farewell’}) }

render() {
        return (
            <div>
                <h3>{this.state.message}</h3>
                <button onClick={this.clickHandler}>Click</button>
            </div>
        )
    }
```

Conditional Rendering

```
class Styles extends React.Component {
    state = {
        marks: 80
    };

    render() {
        let classes = "badge badge-";
        classes += (this.state.marks) > 30 ? "primary" : "warning";

        return (
            <h3>Result:  
                <span className={classes}>
                {this.state.marks > 30 ? 'Pass': 'Fail'}
                </span>
            </h3>
        );
    }
}
```

Runtime class assignment

Conditional Rendering

Conditional Rendering

```
function CheckLogin(props) {
  const isLoggedIn = props.isLoggedIn;
  if (isLoggedIn) {
    return (<h1>Welcome Back!</h1>);
  }
  return (<h1>Please, login.</h1>);
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<CheckLogin isLoggedIn={false} />);
```

List in React

- Lists in React are same as the lists in Javascript/HTML
- We can use lists to show multiple items in a structured manner
- We can use lists for displaying menu, navigation bar etc.
- To traverse a list, we can use map method

```
<ul>
<li> Item 1 </li>
<li> Item 2 </li>
<li> Item 3 </li>
</ul>
//Basic HTML/JS list
```

```
Const Items = [1,2,3,4,5];
Const listItems = Items.map((item)=>
<li>{item}<li>)
Return <ul>{listItems}</ul>
//React list
```

List

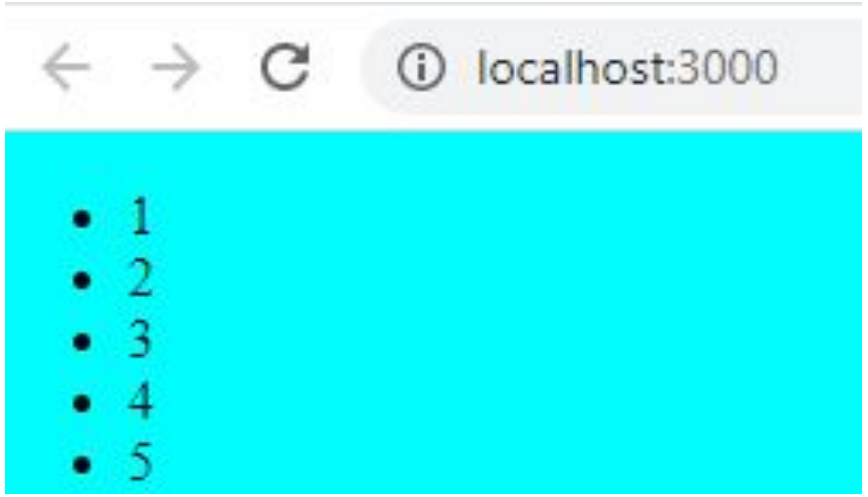
tapp > src > JS App.js > ...

```
import React from 'react'
import ReactDOM from 'react-dom'
class MyList extends React.Component{
    render()
    {
        const list=[1,2,3,4,5]
        const listItems=list.map((itemval) => <li>{itemval}</li>)
        return <ul>{listItems}</ul>
    }
}
export default MyList
```

tapp > src > JS index.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import MyList from './App';
// Here we are passing the component using HTML tag
//name of the component will always start from Capital
ReactDOM.render(<MyList/>, document.getElementById('root'));
```

Map function: it has a call back function which returns individual list elements [1,2…5].



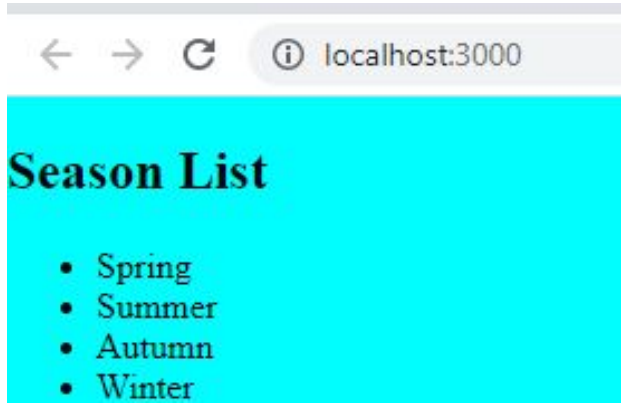
Lists passing data using props

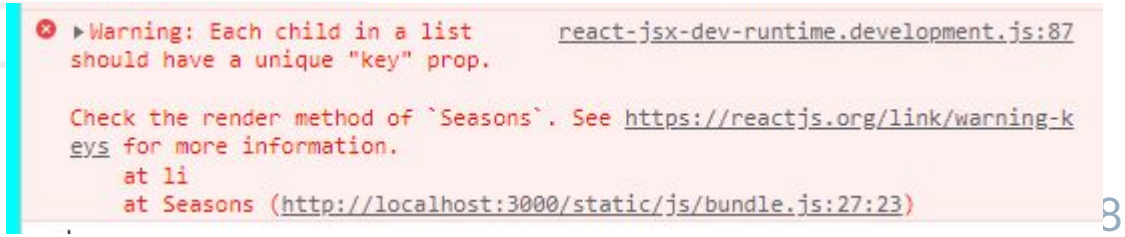
app > src > JS App.js > [@] default

```
import React from 'react'
import ReactDOM from 'react-dom'

function Seasons(props) {
   const s_arr = props.seasons;
   const listItems = s_arr.map((s_arr_item) => <li>{s_arr_item}</li> );
   return (
     <div>
    <h2>Season List</h2>
       <ul>{listItems}</ul>
     </div>
   );
}

export default Seasons;
```

app > src > JS index.js > ...

```
import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import Seasons from './App';

const season_arr = ['Spring', 'Summer', 'Autumn', 'Winter'];

ReactDOM.render(<Seasons seasons={season_arr}/>, document.getElementById('root'));
```





When we run this code we get this warning. Note on developer tool console

Keys

- A key is a special string attribute you need to include when creating list of elements
- Keys help react identify which items have changed, added or deleted
- Keys should be string
- If you choose not to assign an explicit key to list items then React will use index keys by default. You will get a warning
- The above approach has some limitations as using index of items as key may create some issues and give wrong data
- A ‘key’ is the only thing React uses to identify DOM elements
- It is not recommended to use indexes for keys if the order of the item may change

Keys

By default, when recursing on the children of a DOM node, React just iterates over both lists of children at the same time and generates a mutation whenever there's a difference.

```
<ul>
  <li>Duke</li>
  <li>Villanova</li>
</ul>

<ul>
  <li>Connecticut</li>
  <li>Duke</li>
  <li>Villanova</li>
</ul>
```

- React will mutate every child instead of realizing it can keep the Duke and Villanova subtrees intact.
- This inefficiency can be resolved using keys.
- When children have keys, React uses the key to match children in the original tree with children in the subsequent tree.

```
<ul>
  <li key="2015">Duke</li>
  <li key="2016">Villanova</li>
</ul>

<ul>
  <li key="2014">Connecticut</li>
  <li key="2015">Duke</li>
  <li key="2016">Villanova</li>
</ul>
```

Now, React knows that the element with key '2014' is the new one, and the elements with the keys '2015' and '2016' have just moved.

Keys

```
function Seasons(props) {
  const s_arr = props.seasons;
  const listItems = s_arr.map((s_arr_item) => <li key={s_arr_item.id}>{s_arr_item.name}</li> );
  return (
    <div>
    <h2>Season List</h2>
        <ul>{listItems}</ul>
    </div>
  );
}


const season_arr = [
    {id:‘a’, name:'Spring’}, {id:‘b’,name:'Summer’},
    {id:‘c’, name:'Autumn’}, {id:‘d’,name:'Winter’}
];

const root = createRoot(document.getElementById('root'));
root.render(<Seasons seasons={season_arr} />);
```

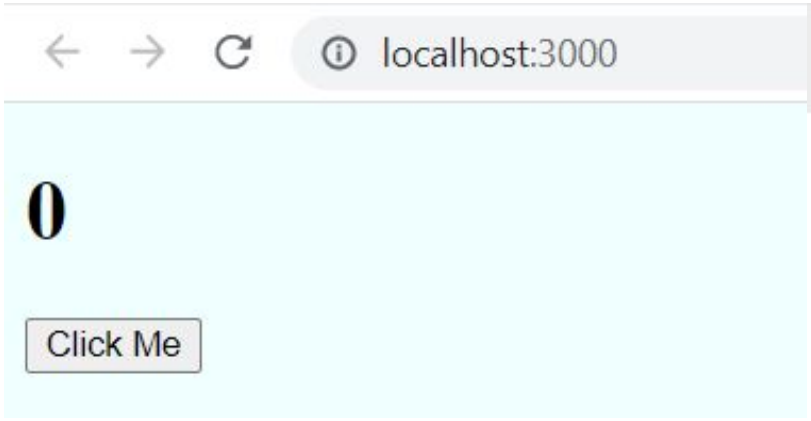
Here original list is a value-id pair. Each item in the array has an id associated with it. Hence, this is the id that is assigned as a key for each item.

Hooks

React Hooks are simple JS functions that let you “hook into” React state and lifecycle features from function components. Hooks can be stateful.

- React 16.8.0 is the first release to support Hooks.
- Hooks allow function components to have access to state and other React features.
- Hooks don’t work inside classes — they let you use React without classes
- Hooks are 100% backwards-compatible.
- React provides a few built-in Hooks like *useState, useEffect* etc.

Example



JS index.js | JS App.js

myreactapp > src > JS App.js > ...

```
1   import React from 'react'
2   import ReactDOM from 'react-dom'
3
4   const IncNum = () =>
5   {
6       console.log("Button is Clicked")
7   }
8   const App = () =>{
9       return(
10          <>
11          <h1>0</h1>
12          <button onClick={IncNum}> Click Me </button>
13          </>
14      )
15  }
16  export default App;
```

Output on console

```
Button is Clicked
>
```

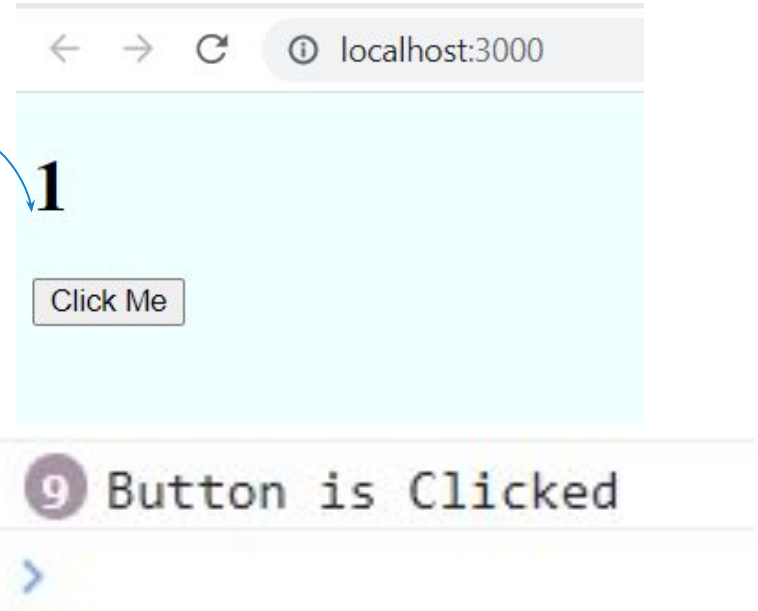
Example

- Suppose we want to increase the value of count

index.js | App.js

myreactapp > src > App.js > ...

```
1   import React from 'react'
2   import ReactDOM from 'react-dom'
3   let count = 1;
4   const IncNum = () =>
5   {
6       count++;
7       console.log("Button is Clicked")
8   }
9   const App = () =>{
10      return(
11          <>
12          <h1>{count}</h1>
13          <button onClick={IncNum}> Click Me </button>
14          </>
15      )
16  }
17  export default App;
```



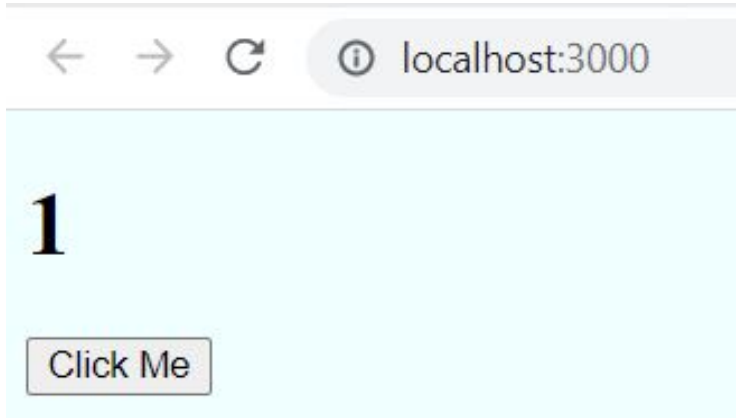
Here the button was clicked 9 times however, no change in the state.

Example

JS index.js JS App.js ×

myreactapp > src > JS App.js > ...

```
1   import React from 'react'
2   import ReactDOM from 'react-dom'
3   let count = 1;
4   const IncNum = () =>
5   {
6       console.log("Button is Clicked: " + count++)
7   }
8   const App = () =>{
9       return(
10          <>
11          <h1>{count}</h1>
12          <button onClick={IncNum}> Click Me </button>
13          </>
14      )
15  }
16  export default App;
```



Here the button was clicked no change in the state, however console displays it correctly

```
Button is Clicked: 1
Button is Clicked: 2
Button is Clicked: 3
Button is Clicked: 4
Button is Clicked: 5
Button is Clicked: 6
Button is Clicked: 7
```

React Hooks

- Prior to the introduction of hooks, React components were written as JavaScript classes. In class components, each component stores all of its state variables in a state property, and state is updated using the setState function.
- Component lifecycle events are handled using methods.
- Props are passed to the component through its constructor() function and the component is rendered in a render() function.
- The use of so many methods to create a simple React component was once a major drawback when using the library.
- Although class components are still a supported feature in React, most developers have opted to use functional components instead since their release in February of 2019.
- Functional components simplify the development process by allowing each component to be created with just a single function.
- This function can take in props as arguments and returns JSX instead of using a separate render() function. React hooks allow functional components to manage state and the component lifecycle in a clean and concise manner.

useState Hook React

- The useState hook allows us to **create state variables for our component.**
- State variables are used to **store dynamic data** in our component which can change as a **user interacts** with it.
- One example of state would be the contents of a **form that the user is filling out.** As they interact with the form fields, the component is continuously **updating its state and re-rendering** in order to keep the form data up to date.

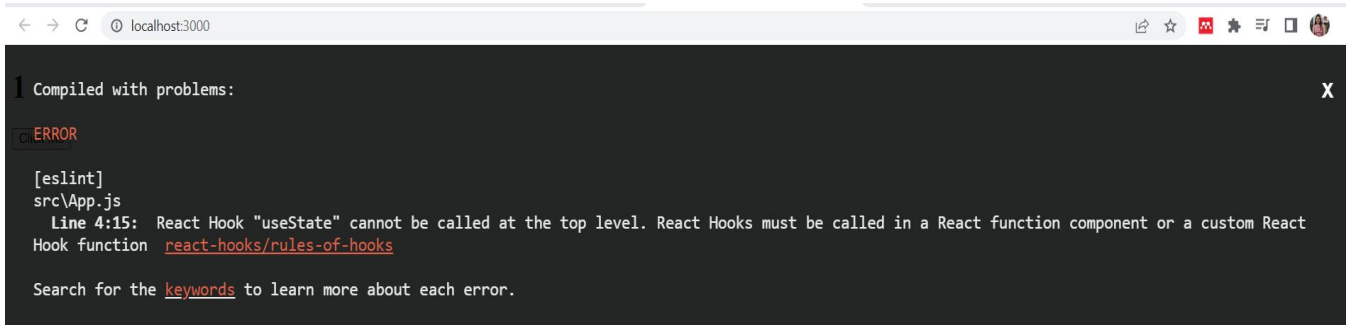
Important Note

- Hooks can be called inside the body of functional components

JS index.js JS App.js

myreactapp > src > JS App.js > ...

```
1  import React, { useState } from 'react'
2  import ReactDOM from 'react-dom'
3
4  const state = useState();
5  console.log(state)
6
7  let count = 1;
8  const IncNum = () =>
9  {
10     console.log("Button is Clicked: " + count++)
11 }
12 const App = () =>{
13     return(
14         <>
15         <h1>{count}</h1>
16         <button onClick={IncNum}> Click Me </button>
17         </>
18     )
19 }
20 export default App;
```



React Hooks

index.js | App.js

myreactapp > src > App.js > ...

```
1  import React, { useState } from 'react'
2  import ReactDOM from 'react-dom'
3
4  const App = () =>{
5  const state = useState();
6  console.log(state)
7
8  let count = 1;
9  const IncNum = () =>
10 {
11     console.log("Button is Clicked: " + count++)
12 }
13     return(
14         <>
15         <h1>{count}</h1>
16         <button onClick={IncNum}> Click Me </button>
17         </>
18     )
19 }
20 export default App;
```

Inside functional components

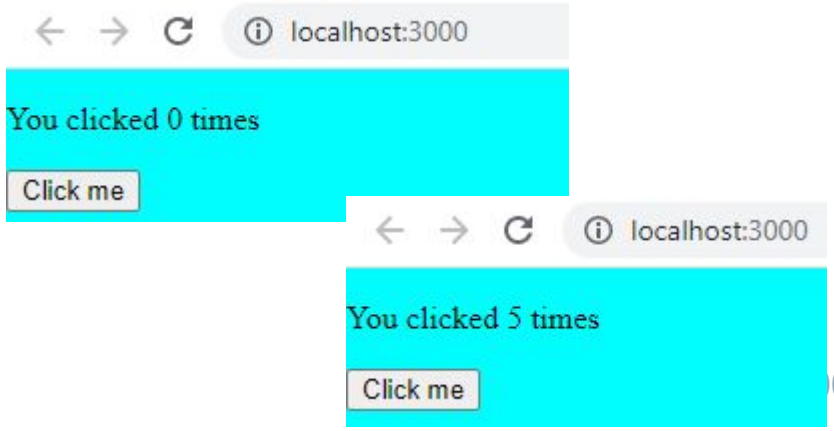
useState Hook

```
import React, { useState } from 'react';
import { createRoot } from 'react-dom/client';

function Example() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}

const root = createRoot(document.getElementById('root'));
root.render(<Example />);
```

To use the *useState* Hook, we first need to import it into our component

We call this a “root” DOM node because everything inside it will be managed by React DOM

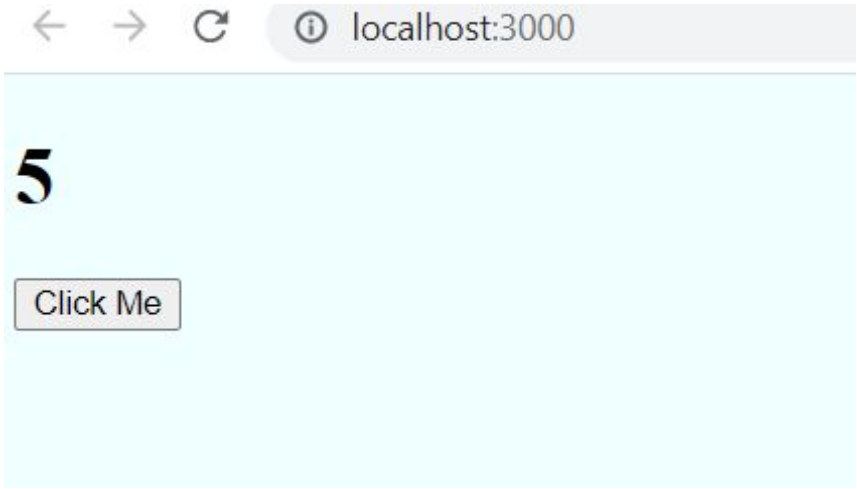
- *useState* declares a state variable.
- The only argument to the *useState*() is the initial state.
- *useState* returns a pair of values: the current state and a function that updates the state.



index.js | App.js

myreactapp > src > App.js > ...

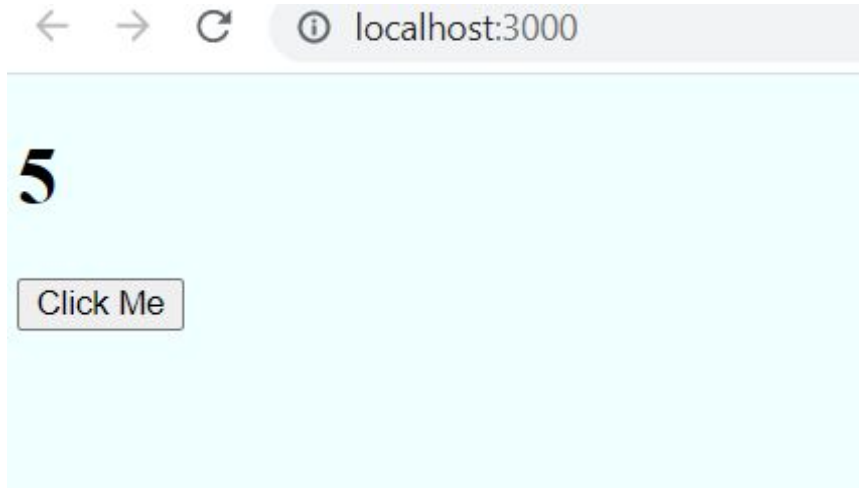
```
1   import React, { useState } from 'react'
2   import ReactDOM from 'react-dom'
3
4   const App = () =>{
5   //returns an array with 2 elements
6   //current data and updated data
7       const state = useState();
8   const [count, setCount]= useState(5);
9
10  const IncNum = () =>
11  {
12      console.log("Button is Clicked: " + count++)
13  }
14      return(
15          <>
16          <h1>{count}</h1>
17          <button onClick={IncNum}> Click Me </button>
18          </>
19      )
20  }
21  export default App;
```

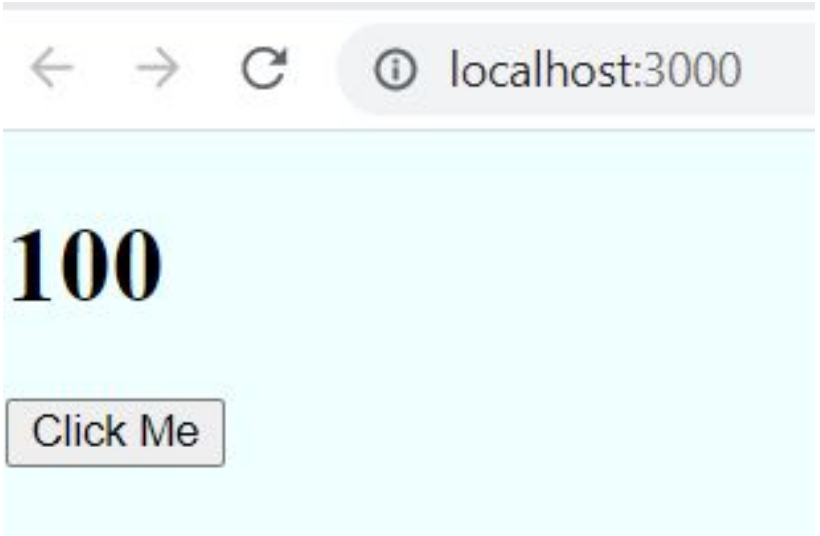


index.js | App.js

myreactapp > src > App.js > ...

```
1   import React, { useState } from 'react'
2   import ReactDOM from 'react-dom'
3
4   const App = () =>{
5   //returns an array with 2 elements
6   //current data and updated data
7       const state = useState();
8   const [count, setCount]= useState(5);
9
10  const IncNum = () =>
11  {
12      setCount(100);
13  }
14      return(
15          <>
16          <h1>{count}</h1>
17          <button onClick={IncNum}> Click Me </button>
18          </>
19      )
20  }
21  export default App;
```

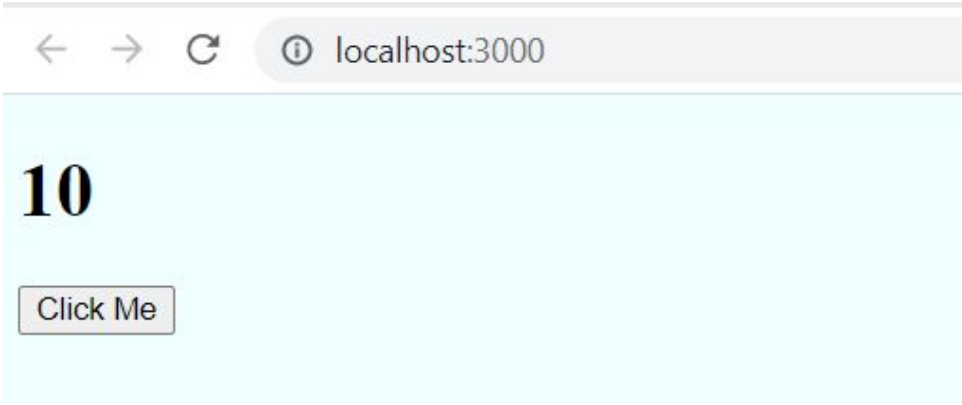


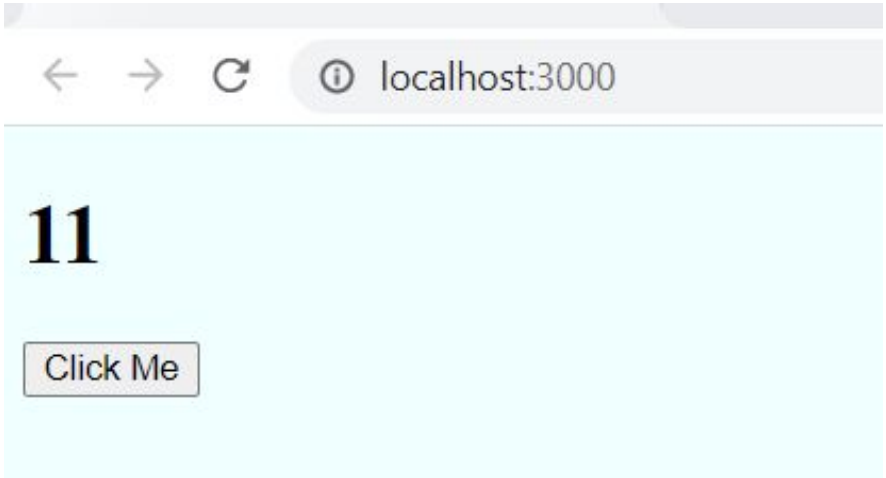


Example

myreactapp > src > JS App.js > ...

```
1   import React, { useState } from 'react'
2   import ReactDOM from 'react-dom'
3
4   const App = () =>{
5   //returns an array with 2 elements
6   //current data and updated data
7       const state = useState();
8   const [count, setCount]= useState(10);
9
10  const IncNum = () =>
11  {
12      setCount(count+1);
13  }
14      return(
15          <>
16          <h1>{count}</h1>
17          <button onClick={IncNum}> Click Me </button>
18          </>
19      )
20  }
21  export default App;
```





useEffect hook

- The useEffect hook allows us to respond to changes in the component lifecycle.
- The component lifecycle refers to a set of events that occur from the time a component is mounted to the DOM until it is removed.
- useEffect is most commonly used to execute code when the component is initially rendered, when it is updated, and when it is unmounted.

useEffect Hook

- *useEffect* lets you perform side effects (like fetching data, directly updating the DOM, and timers) in function components.
- *useEffect* accepts two arguments. The second argument is optional.

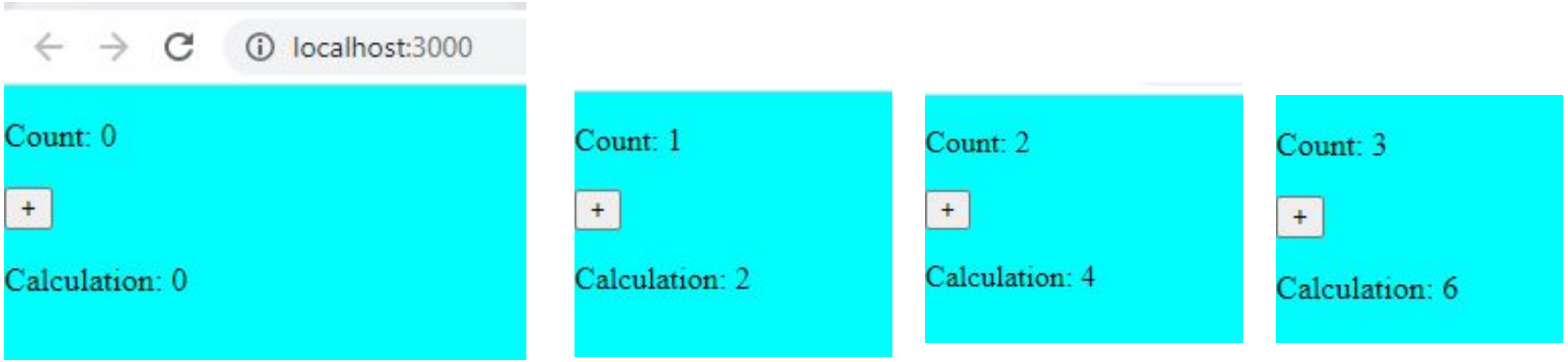
```
import { useState, useEffect } from "react";
import { createRoot } from "react-dom/client";

function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setTimeout(() => {
      setCount((count) => count + 1);
    }, 1000);
  });

  return <h1>I've rendered {count} times!</h1>;
}

const root = createRoot(document.getElementById('root'));
root.render(<Timer />);
```

First import *useEffect* Hook

- *useEffect* tells React that the component needs to do something after render. React will remember the function ("effect") passed, and call it after performing DOM updates.
- *useEffect* runs on every render. But this can be controlled using the second argument of *useEffect* which accepts an array. Pass dependencies to *useEffect* in this array.

Demo Link: https://www.w3schools.com/react/react_useeffect.asp

Practise Example



useEffect Hook

```
import { useState, useEffect } from "react";
import ReactDOM from "react-dom/client";

function Counter() {
  const [count, setCount] = useState(0);
  const [calculation, setCalculation] = useState(0);

  useEffect(() => {
    setCalculation(() => count * 2);
  }, [count]); // <- add the count variable here

  return (
    <>
      <p>Count: {count}</p>
      <button onClick={() => setCount((c) => c + 1)}>+</button>
      <p>Calculation: {calculation}</p>
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Counter />);
```

Demo Link:
https://www.w3schools.com/react/react_useeffect.asp

Forms

- Handling forms is about how you handle the data when it changes value or gets submitted.
- In HTML, form data is usually handled by the DOM.
- In React, form data is usually handled by the components.
- When the data is handled by the components, all the data is stored in the component state.
- You can control changes by adding event handlers in the onChange attribute.
- HTML form elements work a little bit differently from other DOM elements in React, because form elements such as <input>, <textarea> and <select> typically maintain there own state and update it based on user input.
- In React, mutable state is typically kept in the state property of components and only updated with setState()

Controlled Components

- **In a controlled component, form data is handled by a React component.**

React will always have access to component state which reflects the updated value of form elements, state object can be used to submit form data when needed.

```
this.state = {
    email: ''
}

this.changeEmailHandler = (event ) => {
    this.setState({email: event.target.value})
}
```

Values that change within component are called controlled component and handled using state and setState

In the onChange handler we use setState method to update the state , when state gets updated render method is called and new state is assigned as value to input element

In controlled component value of the input field is set to the sate property

onChange event gets fired whenever there is change in the input value

```
<input type='text'  value= {this.state.email}  onChange= {this.changeEmailHandler}  />
```

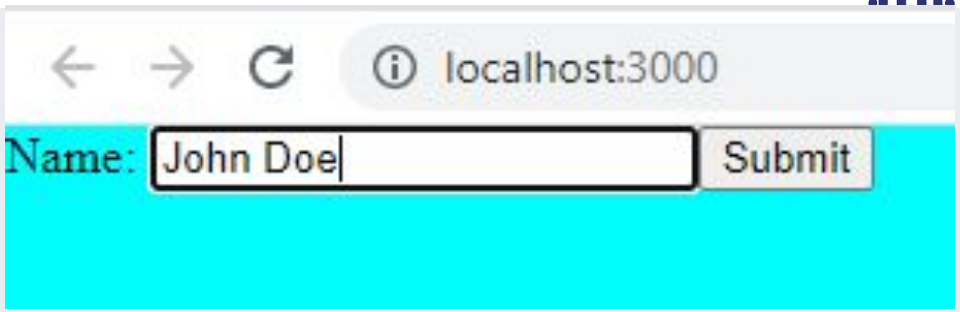
Consider an input element

The input element can have a value

Input element can change its value based on user interaction. Example user can enter email

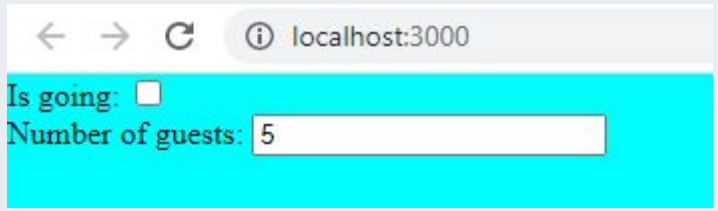
Forms

```
export default class App extends React.Component {
//create component state
constructor() {
    super();
    this.state = { value: "" };
  }
//When you assign a handler to on change event, the event itself is passed as a parameter

handleChange = (event) => { this.setState({ value: event.target.value }); };

  handleSubmit = (event) => {
    alert("A name was submitted: " + this.state.value);
    event.preventDefault();
  };

  render() {
    return (
      <form onSubmit={this.handleSubmit}>
          Name: <input type="text" value={this.state.value} onChange={this.handleChange}  />
        <input type="submit" value="Submit" />
      </form>
    );}}
```



Forms (Handling multiple inputs)

```
export default class App extends React.Component {
  constructor() {
    super();
    this.state = { isGoing: true, numberOfGuests: 2 };
  }

  handleInputChange = (event) => {
    const target = event.target;
    const value = target.type === "checkbox" ? target.checked : target.value;
    const name = target.name;
    this.setState({ [name]: value });
  };

  render() {
    return (
      <form>
        Is going: <input name="isGoing" type="checkbox"
          checked={this.state.isGoing} onChange={this.handleInputChange} />
        <br />
        Number of guests: <input name="numberOfGuests" type="number"
          value={this.state.numberOfGuests} onChange={this.handleInputChange} />
      </form>
    );
  }
}
```



React Router

- **React Router is the de facto standard routing library for React**.
- It conditionally renders components depending on the route being used in the URL (eg. - / for the home page, /about for the about page, etc.).
- **Installation**

  React Router has three modules:

  react-router – core package that provides the core routing components and functions for React Router applications

  react-router-dom – provides the routing components for web development environment [to install npm install react-router-dom]

  react-router-native - provides the routing components for mobile app development

Types of Routers

React Router provides three types of routers:

- **HashRouter** - uses client-side hash routing

  used for static web apps, where the server can only respond to requests for files that it knows about

  useful for situations where you don't have a server logic to handle the client-side

- **BrowserRouter** - uses HTML 5 history API

  should be used when there is a server handling the dynamic requests

- **MemoryRouter** - keeps the URL changes in memory not in the user browsers

  doesn't change the URL in your browser

  useful for testing and non-browser environments like React Native

The URLs they create for a page "about"

```
http://localhost:3000/about        // <BrowserRouter>
http://localhost:3000/#/about      // <HashRouter>
http://localhost:3000          // <MemoryRouter>
```

Link, Route, and Switch components

Route

- Used to render UI component whose path matches the current URL.
- *path* prop is used for specifying the path to be matched.
- / passed as the path to a route component, is called an *Index Route.*
- Route component matches just the beginning of the URL, and not the whole thing.
- *exact* prop is used to render component only if the paths are exactly the same.
- *component* prop is used to specify which component is to be rendered when the path matches the current URL.

Link

- Used for internal linking/ navigation between the pages of the application.
- Link components can be made to point to different routes, with the help of *to* attribute.
- A link component renders an anchor tag in your HTML document.
- *NavLink* is used to add the style attributes to the active routes
- *activeClassName* prop when used with NavLink, applies the corresponding style for an active route.

Switch

- Used to render the components only when path matches. In case of no match it fallbacks to the not found component.
- On rendering, Switch component searches through its children route elements to identify which route's path matches the current URL.
- With <Switch>, only the first child <Route> that matches the location gets rendered. If <Switch> is not used and multiple <Route>s are present, then all the routes that match are rendered inclusively.

Link, Route, and Switch components

```
import React from 'react'
import { Route, Link, Switch } from 'react-router-dom'
import Home from './home.jsx'
import About from './about.jsx'
import Contact from './contact.jsx'
import Notfound from './notfound.jsx'
```

```
class App extends React.Component {
  render() {
    return (
      <div>
        <h2>React Router Demo</h2>
        <nav className="navbar navbar-light">
          <ul className="nav navbar-nav">
            <li> <Link to="/">Home</Link> </li>
            <li> <Link to="/about">About</Link> </li>
            <li> <Link to="/contact">Contact</Link> </li>
          </ul>
        </nav>
        <hr />
        <Switch>
          <Route exact path="/" component={Home} />
          <Route path="/about" component={About} />
          <Route path="/contact" component={Contact} />
          <Route component={Notfound} />
        </Switch>
      </div>
    );
  }
}
```

Useful Reference Links

- https://reactjs.org/
- https://www.taniarascia.com/getting-started-with-react/
- https://www.w3schools.com/react/react_getstarted.asp
- https://www.guru99.com/reactjs-tutorial.html#12
- https://www.tutorialspoint.com/reactjs/index.htm
- https://www.javatpoint.com/reactjs-tutorial
- https://www.youtube.com/watch?v=Ke90Tje7VS0
- https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started